Lecture Notes – Thuật Toán (Algorithms)

Theo phong cách CLRS (Introduction to Algorithms, 4th Ed.)

Tổng hợp từ bài giảng Thuật Toán
Semester 2025–2026
Ngày tạo: 12/03/2026

Abstract

Tài liệu tổng hợp toàn bộ kiến thức môn Thuật Toán theo phong cách sách *Introduction to Algorithms* (CLRS, 4th Edition). Bao gồm: ký hiệu tiệm cận, sắp xếp (Insertion, Selection, Merge, Quick Sort), Master Theorem, lý thuyết đồ thị, Greedy & MST (Kruskal, Prim), Dijkstra, quy hoạch động (Floyd-Warshall, nhân chuỗi ma trận), và P/NP/NP-Complete. Kèm theo bảng cheat sheet, template giải bài tập, và phụ lục tra cứu ký hiệu toán học.

Keywords Thuật toán · CLRS · Sắp xếp · Đồ thị · MST · Dijkstra · Quy hoạch động · NP-Complete

Cách sử dụng tài liệu này – Hệ thống tự học

Tài liệu này không chỉ để đọc – mà để **tương tác**. Bạn sẽ gặp 4 loại hộp đặc biệt:

- **Tại sao?** – Dừng lại và suy nghĩ trước khi đọc tiếp. Tự trả lời trước, rồi kiểm tra.
- **Tự Trace** – Lấy giấy bút, tự trace thuật toán bằng tay. **Đừng chỉ đọc!**
- **Giải thích bằng lời bạn** – Thử giải thích như đang dạy bạn bè. Nếu bí = bạn chưa hiểu.
- **Tự kiểm tra** – Câu hỏi nhanh. Trả lời trong đầu trước khi lật trang.

KHUNG NHẬN DẠNG ĐỀ THI NHANH

Khi đề bài yêu cầu...	Dùng kỹ thuật	Chương
Sắp xếp mảng, so sánh thuật toán sort	**Sorting Algorithms**	Ch.2
Giải công thức đệ quy T(n)	**Master Theorem / Recursion Tree**	Ch.3
Biểu diễn đồ thị, danh sách kề, ma trận kề	**Graph Representation**	Ch.4
Tìm cây bao trùm nhỏ nhất (MST)	**Kruskal / Prim**	Ch.5
Tìm đường đi ngắn nhất từ 1 đỉnh	**Dijkstra**	Ch.6
Tìm đường đi ngắn nhất mọi cặp đỉnh	**Floyd-Warshall**	Ch.7
Tính Fibonacci, tổ hợp C(n,k)	**Dynamic Programming**	Ch.7
Nhân chuỗi ma trận tối ưu	**Matrix Chain Multiplication**	Ch.8
Chứng minh bài toán NP-Complete	**P, NP, NP-Complete**	Ch.9

Table 1: Khung nhận dạng đề thi nhanh – Quick Exam Pattern Recognition

---

1 Chương 1: Nền tảng – Foundations (CLRS Ch.1–3)

Trực giác: Trước khi viết 1 dòng code, cần có ngôn ngữ chung để **so sánh** hiệu quả thuật toán – đó là ký hiệu tiệm cận (O, Θ, Ω). Cũng cần công cụ **chứng minh đúng** – đó là Loop Invariant. Fibonacci là bài toán mẫu để thấy sức mạnh của DP so với đệ quy ngây thơ.

1.1 1.1 Thuật toán là gì?

Định nghĩa (CLRS): Thuật toán là một thủ tục tính toán xác định rõ ràng, nhận **input** và cho ra **output**, giải quyết một **bài toán tính toán** cụ thể.

Tính đúng đắn (Correctness): Thuật toán **đúng** nếu với **mọi** input hợp lệ, nó **dừng** và cho ra output đúng.

1.2 1.2 Ký hiệu tiệm cận (Asymptotic Notation) – CLRS Ch.3

1.2.1 Định nghĩa chính xác:

Θ (Theta) – Chặn chặt:

$f(n) = \Theta(g(n)) \Leftrightarrow \exists\, c_1, c_2 > 0$, n_0 sao cho $\forall n \geq n_0$:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

O (Big-O) – Chặn trên:

$f(n) = O(g(n)) \Leftrightarrow \exists\, c > 0$, n_0 sao cho $\forall n \geq n_0$:

$$f(n) \leq c \cdot g(n)$$

Ω (Omega) – Chặn dưới:

$f(n) = \Omega(g(n)) \Leftrightarrow \exists\, c > 0$, n_0 sao cho $\forall n \geq n_0$:

$$f(n) \geq c \cdot g(n)$$

o (little-o) – Chặn trên không chặt:

$f(n) = o(g(n)) \Leftrightarrow \lim_{n\to\infty} f(n)/g(n) = 0$

ω (little-omega) – Chặn dưới không chặt:

$f(n) = \omega(g(n)) \Leftrightarrow \lim_{n\to\infty} f(n)/g(n) = \infty$

1.2.2 Quan hệ:

$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n))$ **VÀ** $f(n) = \Omega(g(n))$

1.2.3 Ví dụ:

- $2n^2 + 3n + 1 = \Theta(n^2)$ – chọn $c_1 = 2, c_2 = 6, n_0 = 1$
- $n \lg n = O(n^2)$ – vì $n \lg n \leq n^2$ với $n \geq 1$
- $n^3 = \Omega(n^2)$ – vì $n^3 \geq n^2$ với $n \geq 1$

? Tại sao? Tại sao ta không dùng **đo thời gian chạy thực tế** (empirical timing) thay cho Big-O? Gợi ý: nghĩ về sự khác nhau giữa các máy tính, kích thước input, và tính phổ quát (generality).

⇒ Giải thích bằng lời bạn: Giải thích sự khác biệt giữa O, Θ, và Ω cho một người bạn không học CS. Dùng ví dụ thực tế (tốc độ xe, giá cả...). Nếu bạn không giải thích được Θ = "chặn chặt" nghĩa là gì – bạn chưa thực sự hiểu.

1.3 1.2a Cách ước lượng Complexity từ Pseudocode

Trực giác: Nhìn pseudocode, ta "đếm" số phép toán cơ bản (so sánh, gán, cộng) theo kích thước input n. Không cần đếm chính xác – chỉ cần xác định **bậc tăng trưởng** (growth rate). Dưới đây là các quy tắc nhanh giúp ước lượng ngay từ cấu trúc code.

1.3.1 Quy tắc ước lượng Time Complexity:

1. **Câu lệnh đơn** (gán, so sánh, return): $\Theta(1)$ – hằng số.
2. **Vòng lặp đơn** `for i = 1 to n`: thân vòng chạy n lần $\to \Theta(n)$.
3. **Vòng lặp lồng nhau** (nested loops):
   - 2 vòng `for` lồng nhau, mỗi vòng n lần $\to n \times n = \Theta(n^2)$. *(Bubble Sort, Selection Sort)*
   - 3 vòng lồng $\to \Theta(n^3)$. *(Floyd-Warshall, Matrix Chain)*
4. **Vòng lặp chia đôi** `while n > 1: n = n/2`: chạy $\lg n$ lần $\to \Theta(\lg n)$. *(Binary Search)*
5. **Đệ quy chia đôi + gộp tuyến tính:**
   $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \lg n)$. *(Merge Sort)*
   Dùng **Master Theorem** (Chương 3) hoặc **Recursion Tree** để giải.
6. **Đệ quy phân nhánh đầy đủ:**
   $T(n) = 2T(n-1) + \Theta(1) \to \Theta(2^n)$. *(Fibonacci naive)*

Bảng tra nhanh: Cấu trúc code → Complexity

Cấu trúc Pseudocode	Time	Ví dụ
1 vòng `for` (1 đến n)	$\Theta(n)$	Tìm max
2 vòng `for` lồng nhau	$\Theta(n^2)$	Bubble Sort
3 vòng `for` lồng nhau	$\Theta(n^3)$	Floyd-Warshall
`while n = n/2`	$\Theta(\lg n)$	Binary Search
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \lg n)$	Merge Sort
$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$	Quick Sort (worst)
$T(n) = 2T(n-1) + \Theta(1)$	$\Theta(2^n)$	Fibonacci (naive)

1.3.2 Quy tắc ước lượng Space Complexity:

1. **Biến đơn** (i, j, key): $O(1)$ – không phụ thuộc n.
2. **Mảng phụ** kích thước n (ví dụ: mảng tạm trong Merge): $\Theta(n)$.
3. **Stack đệ quy:** Mỗi lần gọi đệ quy chiếm $O(1)$ stack frame. Chiều sâu đệ quy $d \to O(d)$ space.
   - Merge Sort: sâu $\lg n$ tầng + mảng tạm $\Theta(n) \to \Theta(n)$.
   - Quick Sort: sâu $\lg n$ (avg) hoặc n (worst) $\to O(\lg n)$ đến $O(n)$.
4. **Ma trận** $n \times n$: $\Theta(n^2)$. *(Floyd-Warshall: ma trận D)*

1.4 1.2b Độ phức tạp không gian (Space Complexity)

Trực giác: Time Complexity hỏi "mất **bao lâu**?". Space Complexity hỏi "tốn **bao nhiêu bộ nhớ**?". Trong thực tế, bộ nhớ thường là tài nguyên **hữu hạn hơn** thời gian – bạn có thể chờ lâu hơn, nhưng không thể mua thêm RAM giữa lúc chạy. Hiểu space complexity giúp chọn đúng thuật toán cho bài toán lớn.

1.4.1 Hai loại bộ nhớ cần phân biệt:

1. **Auxiliary Space (Bộ nhớ phụ):** Bộ nhớ **thêm** ngoài input. Ví dụ: mảng tạm trong Merge Sort.
2. **Total Space:** Auxiliary + Input. Ví dụ: ma trận $D[n \times n]$ trong Floyd-Warshall là *cả* output lẫn workspace.

   Khi nói "Space Complexity = $O(n)$" thường ám chỉ **auxiliary space**, trừ khi nói rõ.

1.4.2 In-place là gì?

In-place = thuật toán **chỉ dùng** $O(1)$ **hoặc** $O(\lg n)$ **bộ nhớ phụ**, tức chỉnh sửa trực tiếp trên input mà không tạo bản sao lớn.

- In-place: Insertion Sort, Quick Sort, Selection Sort
- **Not** in-place: Merge Sort (cần mảng tạm $\Theta(n)$)

1.4.3 Stack Space cho đệ quy:

Mỗi lần gọi đệ quy tạo **stack frame** mới trên bộ nhớ. Thuật toán đệ quy sâu d mức tiêu tốn $O(d)$ stack space.

- Quick Sort: chiều sâu trung bình $O(\lg n)$, worst case $O(n)$ (nên Random + tail-call optimization)
- Merge Sort: luôn $O(\lg n)$ chiều sâu, nhưng cần $\Theta(n)$ mảng phụ bên ngoài
- Fibonacci đệ quy: chiều sâu $O(n) \to$ stack space $O(n)$; DP bottom-up chỉ $O(1)$

1.4.4 Bảng Space Complexity chi tiết:

> **Lưu ý thuật ngữ – Auxiliary (Bộ nhớ phụ):** Cột này chỉ đánh giá lượng bộ nhớ *sinh thêm* trong quá trình chạy (biến tạm, mảng copy, dung lượng ngăn xếp đệ quy...), **hoàn toàn không tính** tới kích thước lưu trữ của bản thân dữ liệu Input (V, E, n). Điều này giúp ta thấy rõ chi phí thực sự dội lên bộ nhớ của từng thuật toán.

Thuật toán	Auxiliary	In-place?	Tại sao?
Insertion Sort	$O(1)$	Có	Chỉ dùng biến `key`, `i` – dịch phần tử trên chính mảng
Selection Sort	$O(1)$	Có	Chỉ dùng biến `min`, `i`, `j` – swap trên chính mảng
Merge Sort	$\Theta(n)$	**Không**	Hàm Merge tạo mảng tạm $L[1..n_1]$, $R[1..n_2]$ để trộn
Quick Sort	$O(\lg n)$ avg	Có	Partition chỉ swap trên mảng. $O(\lg n)$: stack đệ quy
Kruskal	$O(V)$	–	Union-Find: mảng parent[] + rank[] cho V đỉnh
Prim	$O(V)$	–	Priority queue cho V đỉnh: key[] + π[]
Dijkstra	$O(V)$	–	Mảng $d[V]$, $\pi[V]$, priority queue Q
Floyd-Warshall	$\Theta(V^2)$	–	Ma trận $D[V \times V]$ lưu khoảng cách
MCM	$\Theta(n^2)$	–	Bảng $m[n \times n] + s[n \times n]$
Pascal $C(n,k)$ v1	$O(nk)$	–	Bảng 2D $C[0..n][0..k]$
Pascal $C(n,k)$ v2	$O(k)$	–	Chỉ dùng 1 hàng $C[0..k]$, cập nhật **ngược**
Fibonacci DP	$O(1)$	–	Chỉ lưu a, b, c (3 biến)
Fibonacci Matrix	$O(1)$	–	Ma trận 2×2 cố định

Table 2: So sánh Space Complexity chi tiết các thuật toán sắp xếp

> **? Tại sao?** Tại sao Merge Sort cần $\Theta(n)$ extra space mà Quick Sort thì không? Gợi ý: nghĩ về cách Merge gộp 2 mảng con – có thể merge "tại chỗ" (in-place) không? Nếu có, tại sao thực tế không ai làm vậy?

> **✓ Tự kiểm tra:** Quick Sort có phải "in-place" hoàn toàn không? Hay vẫn tốn bộ nhớ? (Gợi ý: nghĩ về stack đệ quy.) Worst case stack depth của Quick Sort là bao nhiêu? Làm sao tránh?

1.5 Chứng minh tính đúng đắn – Loop Invariant (CLRS §2.1)

1.5.1 Ba bước chứng minh:

1. **Initialization (Khởi tạo):** Bất biến đúng **trước lần lặp đầu tiên**
2. **Maintenance (Duy trì):** Nếu đúng trước lần lặp thứ $i \to$ đúng trước lần lặp thứ $i + 1$

3. **Termination (Kết thúc):** Khi vòng lặp dừng, bất biến + điều kiện dừng → **kết quả đúng**

   Tương tự **quy nạp toán học**, nhưng có bước dừng → cho kết luận hữu ích.

▷ Tự Trace: Loop Invariant – Điền 3 bước

Hãy tự chứng minh Insertion Sort đúng bằng Loop Invariant. Điền vào:
1. Bất biến: "Tại đầu mỗi lần lặp for với j, _____"
2. Initialization ($j =$ _____): _____
3. Maintenance: Giả sử đúng trước lần j, chứng minh đúng trước lần $j+1$:
_____
4. Termination: Vòng lặp dừng khi $j =$ _____. Kết luận: _____
(So sánh với phần 1.3 ở trên sau khi hoàn thành.)

1.6 1.4 Fibonacci – Bốn cách tiếp cận (CLRS Problem 31-3 style)

1.6.1 Cách 1: Đệ quy – $O(2^n)$

Pseudocode 1: Fibonacci – Đệ quy

```
Fib(n):
    if n <= 1: return n
    return Fib(n-1) + Fib(n-2)
```

Phân tích Complexity – Fibonacci Đệ quy

Dòng code	Số lần chạy	Chi phí
`if n <= 1: return n`	$\Theta(1)$	base case
`return Fib(n-1) + Fib(n-2)`	$T(n-1)+T(n-2)$	2 nhánh đệ quy

Recurrence: $T(n) = T(n-1) + T(n-2) + \Theta(1)$.
Time: Cây đệ quy có $\sim 2^n$ nút (thực tế $\Theta(\varphi^n)$ với $\varphi \approx 1.618$) → **exponential**.
Space: Chiều sâu cây $= n$ → stack đệ quy $O(n)$.

Phân tích (CLRS style): $T(n) = T(n-1) + T(n-2) + \Theta(1)$. Cây đệ quy có $\sim 2^n$ nút → $T(n) = \Theta(\varphi^n)$ với $\varphi = (1+\sqrt{5})/2 \approx 1.618$ (tỷ lệ vàng).

1.6.2 Cách 2: DP Bottom-up – $O(n)$ time, $O(n)$ space

Pseudocode 2: Fibonacci – DP Bottom-up

```
Fib2(n):
    F[0] = 0, F[1] = 1
    for i = 2 to n:
        F[i] = F[i-1] + F[i-2]
    return F[n]
```

Phân tích Complexity – Fibonacci DP Bottom-up

Dòng code	Số lần chạy	Chi phí
`F[0]=0, F[1]=1`	2	$\Theta(1)$ khởi tạo
`for i = 2 to n`	$n-1$	1 vòng lặp
`  F[i] = F[i-1] + F[i-2]`	$n-1$	$\Theta(1)$ mỗi lần
`return F[n]`	1	$\Theta(1)$

Time: 1 vòng `for` chạy $n-1$ lần → $\Theta(n)$. *Nhanh gấp cấp số mũ so với đệ quy!*
Space: Mảng $F[0..n]$ có $n+1$ phần tử → $\Theta(n)$.

Tại sao nhanh hơn? Mỗi bài toán con $F[i]$ chỉ tính **1 lần** → tổng $\Theta(n)$ phép cộng.

1.6.3 Cách 3: DP tối ưu space – $O(n)$ time, $O(1)$ space

Pseudocode 3: Fibonacci – DP tối ưu space

```
Fib3(n):
    if n <= 1: return n
    a := 0, b := 1
    for i = 2 to n:
        c := a + b; a := b; b := c
    return c
```

Phân tích Complexity – Fibonacci DP tối ưu space

Dòng code	Số lần chạy	Chi phí
`a := 0, b := 1`	2	$\Theta(1)$ khởi tạo
`for i = 2 to n`	$n-1$	1 vòng lặp
`c:=a+b; a:=b; b:=c`	$n-1$	$\Theta(1)$ mỗi lần
`return c`	1	$\Theta(1)$

Time: Giống cách 2: 1 vòng `for` $\to \Theta(n)$.
Space: Chỉ dùng 3 biến $a, b, c \to O(1)$! *Tối ưu hơn cách 2 (từ $\Theta(n)$ xuống $O(1)$).*

1.6.4 Cách 4: Lũy thừa ma trận – $O(\lg n)$

Dựa trên công thức:

$$\begin{pmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

Dùng **repeated squaring** (CLRS §31.6): tính A^n bằng cách bình phương liên tiếp $\to O(\lg n)$ phép nhân ma trận 2×2.

---

2 Chương 2: Thuật toán Sắp xếp – Sorting Algorithms (CLRS Ch.2, 7, 8)

Trực giác: Sắp xếp là bài toán nền tảng nhất của CS. Chúng ta học 4 thuật toán không phải để nhớ code, mà để hiểu **3 mô hình thiết kế**: (1) Incremental (Insertion Sort), (2) Divide & Conquer (Merge/Quick Sort), và (3) giới hạn lý thuyết $\Omega(n \lg n)$. Trong thực tế, Quick Sort **nhanh nhất** dù worst case $\Theta(n^2)$, vì pivot ngẫu nhiên hiếm khi xấu.

Sai lầm thường gặp: (1) Nhầm worst case của Quick Sort với average case – trong thực tế dùng randomized nên average $\Theta(n \lg n)$. (2) Merge Sort cần thêm $O(n)$ bộ nhớ – không phải in-place. (3) Selection Sort tuy $O(n^2)$ nhưng không stable – không thể dùng khi thứ tự relative quan trọng.

2.1 2.1 Bảng tổng hợp

Thuật toán	Time Best	Time Avg	Time Worst	Space	Stable	In-place
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No	Yes
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n)$	Yes	No
Quick Sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n^2)$	$O(\lg n)$	No	Yes
Heap Sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(1)$	No	Yes

Thuật toán	Time Best	Time Avg	Time Worst	Space	Stable	In-place
Counting Sort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	Yes	No

Table 3: Bảng tổng hợp Complexity các thuật toán sắp xếp

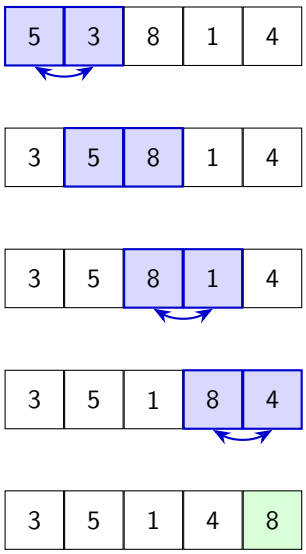
2.2 2.2 Bubble Sort

2.2.1 Ý tưởng

Bubble Sort (Sắp xếp nổi bọt) ít dùng thực tế nhưng là cách dễ nhất để làm quen với hình ảnh hoá thuật toán sắp xếp (giống các video hoạt họa). So sánh từng cặp phần tử kề nhau, nếu sai thứ tự thì đổi chỗ (swap). Phần tử lớn nhất sẽ "nổi" lên cuối mảng.

2.2.2 Minh họa Animation

Warm-up: Minh họa vòng lặp đầu tiên



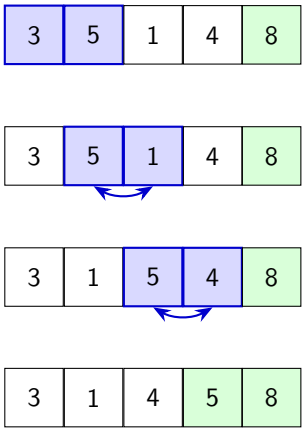
1. So sánh $5 > 3 \rightarrow$ Swap (Đổi chỗ)
2. So sánh $5 < 8 \rightarrow$ Giữ nguyên
3. So sánh $8 > 1 \rightarrow$ Swap
4. So sánh $8 > 4 \rightarrow$ Swap

Cuối vòng 1: Số lớn nhất (8) đã về đúng vị trí (màu xanh)

Figure 1: Bubble Sort – Minh họa vòng lặp đầu tiên trên mảng $[5, 3, 8, 1, 4]$

Minh họa vòng lặp thứ hai

Mảng sau vòng 1: $[3, 5, 1, 4, 8]$. Phần tử cuối (8) đã cố định. Chỉ duyệt 4 phần tử đầu.



1. So sánh $3 < 5 \rightarrow$ Giữ nguyên
2. So sánh $5 > 1 \rightarrow$ Swap
3. So sánh $5 > 4 \rightarrow$ Swap

Cuối vòng 2: Hai phần tử lớn nhất (5, 8) đã đúng vị trí

Figure 2: Bubble Sort – Minh họa vòng lặp thứ hai trên mảng $[3, 5, 1, 4, 8]$

Pseudocode 3a: Bubble Sort

```
BubbleSort(A, n):
    for i = 1 to n-1:
        for j = 1 to n-i:
            if A[j] > A[j+1]:
                swap(A[j], A[j+1])
```

Phân tích Complexity – Bubble Sort

Dòng code	Số lần chạy	Chi phí
`for i = 1 to n-1`	$n-1$	vòng ngoài
`for j = 1 to n-i`	$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$	vòng trong
`if A[j]>A[j+1]: swap`	$\frac{n(n-1)}{2}$	$\Theta(1)$ mỗi lần

Time: $T(n) = \frac{n(n-1)}{2} = \Theta(n^2)$ (worst/avg). Best $\Theta(n)$ nếu thêm cờ early stop.
Space: $O(1)$ – chỉ dùng biến i, j, swap tại chỗ (in-place).

2.2.3 Complexity

Worst/Average $= \Theta(n^2)$. Best $= \Theta(n)$ nếu thêm cờ "đã sắp" (early termination). Space $= O(1)$ (in-place). Stable.

2.2.4 Loop Invariant

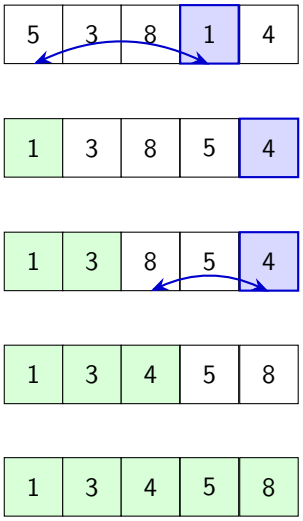
- **Bất biến:** Sau vòng lặp ngoài thứ i, i phần tử lớn nhất đã nằm đúng vị trí cuối mảng $A[n-i+1..n]$, đã sắp.
- **Init ($i = 1$):** Chưa vòng nào chạy, chưa phần tử nào "cố định" – bất biến đúng (trivial).
- **Maintenance:** Vòng i so sánh các cặp kề $A[j], A[j+1]$. Swap nếu sai thứ tự $\rightarrow$ phần tử lớn nhất trong $A[1..n-i+1]$ "nổi" lên vị trí $n-i+1$. Sau vòng i: i phần tử lớn nhất đã cố định.
- **Termination ($i = n-1$):** $n-1$ phần tử lớn nhất đã đúng chỗ $\rightarrow$ phần tử còn lại (nhỏ nhất) cũng đúng $\rightarrow$ mảng sắp. $\square$

2.3 2.3 Selection Sort

2.3.1 Ý tưởng

Duyệt mảng từ trái sang phải. Tại mỗi vị trí i, **tìm phần tử nhỏ nhất** trong phần chưa sắp $A[i..n]$, rồi **swap** nó về vị trí i. Sau mỗi vòng, phần đầu mảng $A[1..i]$ đã sắp đúng chỗ. Đơn giản nhưng luôn $\Theta(n^2)$ vì phải quét toàn bộ phần còn lại mỗi lần.

2.3.2 Minh họa Animation



Vòng 1: Tìm min trong $A[1..5] = 1$ (vị trí 4). Swap $A[1] \leftrightarrow A[4]$

Vòng 2: Tìm min trong $A[2..5] = 3$ (đã đúng vị trí). Không swap.

Vòng 3: Tìm min trong $A[3..5] = 4$ (vị trí 5). Swap $A[3] \leftrightarrow A[5]$

Vòng 4: Tìm min trong $A[4..5] = 5$ (đã đúng). Không swap.

Mảng đã được sắp xếp hoàn toàn! $[1, 3, 4, 5, 8]$

Figure 3: Selection Sort – Toàn bộ quá trình sắp xếp trên mảng $[5, 3, 8, 1, 4]$

Pseudocode 3b: Selection Sort

```
SelectionSort(A, n):
    for i = 1 to n-1:
        min_idx = i
        for j = i+1 to n:
            if A[j] < A[min_idx]:
                min_idx = j
        swap(A[i], A[min_idx])
```

Phân tích Complexity – Selection Sort

Dòng code	Số lần chạy	Chi phí
`for i = 1 to n-1`	$n-1$	vòng ngoài
`  min_idx = i`	$n-1$	$\Theta(1)$
`  for j = i+1 to n`	$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$	tìm min
`  swap(A[i], A[min_idx])`	$n-1$	$\Theta(1)$

Time: $T(n) = \frac{n(n-1)}{2} = \Theta(n^2)$ mọi trường hợp (luôn duyệt hết).
Space: $O(1)$ – chỉ dùng biến $i, j, \text{min_idx}$ (in-place).

2.3.3 Complexity

Worst/Average/Best $= \Theta(n^2)$ (luôn quét toàn bộ phần còn lại). Space $= O(1)$ (in-place). **Không stable** (swap có thể đổi thứ tự tương đối).

2.3.4 Loop Invariant

- **Bất biến:** Tại đầu vòng lặp i, mảng con $A[1..i-1]$ chứa $i-1$ phần tử nhỏ nhất của mảng, đã sắp tăng dần.
- **Init ($i = 1$):** $A[1..0]$ rỗng – bất biến đúng (trivial).
- **Maintenance:** Tìm min trong $A[i..n]$, swap về vị trí i. $A[i]$ giờ là phần tử nhỏ nhất trong phần còn lại, và $\geq$ tất cả $A[1..i-1] \to A[1..i]$ đã sắp.
- **Termination ($i = n$):** $A[1..n-1]$ chứa $n-1$ phần tử nhỏ nhất, đã sắp. Phần tử $A[n]$ là lớn nhất $\to$ mảng sắp. $\square$

2.4 2.4 Insertion Sort – CLRS §2.1 (Chứng minh đầy đủ)

2.4.1 Ý tưởng

Giống cách xếp bài trên tay: lấy từng lá bài (phần tử $A[j]$) và **chèn** vào đúng vị trí trong phần đã sắp $A[1..j-1]$. Dịch các phần tử lớn hơn sang phải để tạo chỗ. Hiệu quả trên mảng **gần sắp** (best case $\Theta(n)$), nhưng worst case $\Theta(n^2)$.

Pseudocode 4: Insertion Sort

```
InsertionSort(A, n):
    for j = 2 to n:
        key = A[j]
        // Chen A[j] vao day da sap A[1..j-1]
        i = j - 1
        while i > 0 and A[i] > key:
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

Phân tích Complexity – Insertion Sort (CLRS §2.2)

Dòng code	Số lần chạy	Chi phí
`for j = 2 to n`	$n-1$	vòng ngoài
`key = A[j]`	$n-1$	$\Theta(1)$
`i = j - 1`	$n-1$	$\Theta(1)$
`while i>0 and A[i]>key`	$\sum t_j$	t_j = số dịch ở bước j
`A[i+1] = A[i]; i--`	$\sum(t_j - 1)$	$\Theta(1)$ mỗi lần
`A[i+1] = key`	$n-1$	$\Theta(1)$

Time Best (đã sắp): $t_j = 1\ \forall j \to$ while không chạy $\to T(n) = \Theta(n)$.
Time Worst (sắp ngược): $t_j = j \to \sum_{j=2}^{n} j = \frac{n(n-1)}{2} \to T(n) = \Theta(n^2)$.
Space: $O(1)$ – chỉ dùng biến key, i, j (in-place).

2.4.2 Chứng minh đúng đắn (Loop Invariant – CLRS style):

Loop Invariant: Tại đầu mỗi lần lặp for với j, mảng con $A[1..j-1]$ chứa các phần tử **ban đầu** của $A[1..j-1]$ nhưng **đã được sắp xếp** tăng dần.

Initialization ($j = 2$): $A[1..1]$ chỉ có 1 phần tử $\to$ đã sắp. (OK)

Maintenance: Giả sử $A[1..j-1]$ đã sắp. Vòng while dịch các phần tử $>$ key sang phải, tìm vị trí đúng cho key. Sau khi chèn, $A[1..j]$ đã sắp. (OK)

Termination: Vòng for dừng khi $j = n+1$. Bất biến cho biết $A[1..n]$ đã sắp = **toàn bộ mảng**. (OK)

2.4.3 Phân tích Complexity (CLRS §2.2):

- **Best case:** Mảng đã sắp $\to$ while không chạy $\to$ mỗi j tốn $\Theta(1) \to T(n) = \Theta(n)$

- **Worst case:** Mảng sắp ngược → while chạy $j-1$ lần → $T(n) = \Theta(n^2)$
- Chính xác: $T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum t_j + c_6 \sum (t_j - 1) + c_7 \sum (t_j - 1) + c_8(n-1)$

2.4.4 Minh họa Animation

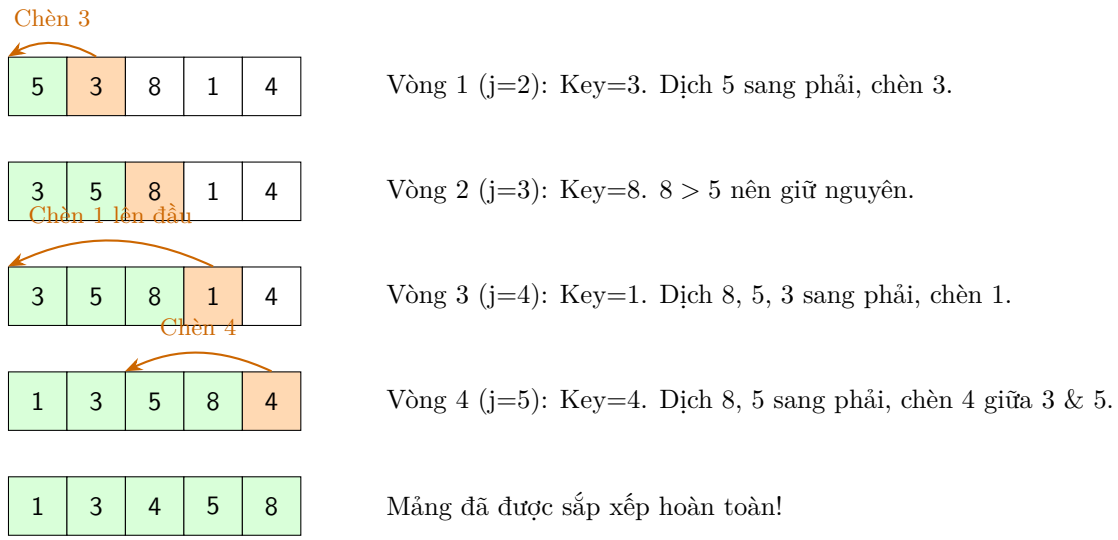


Figure 4: Insertion Sort – Minh họa từng bước chèn trên mảng $[5, 3, 8, 1, 4]$

2.5 2.5 Merge Sort – CLRS §2.3 (Divide & Conquer)

2.5.1 Ý tưởng

Áp dụng Divide & Conquer: chia mảng thành 2 nửa bằng nhau, đệ quy sắp xếp từng nửa, rồi **trộn (merge)** 2 nửa đã sắp thành mảng hoàn chỉnh. Công việc nặng nằm ở bước *trộn*, cần mảng tạm $O(n)$. Luôn $\Theta(n \lg n)$ bất kể input, nhưng **không in-place** (tốn thêm bộ nhớ).

Mô hình D&C (CLRS §2.3.1):

1. **Divide:** Chia mảng thành 2 nửa tại $q = \lfloor (p+r)/2 \rfloor$
2. **Conquer:** Gọi đệ quy MergeSort trên mỗi nửa
3. **Combine:** Trộn (Merge) 2 nửa đã sắp

Pseudocode 5: Merge Sort

```
MergeSort(A, p, r):
    if p >= r: return
    q = floor((p + r) / 2)
    MergeSort(A, p, q)
    MergeSort(A, q+1, r)
    Merge(A, p, q, r)

Merge(A, p, q, r):
    n1 = q - p + 1,  n2 = r - q
    Tao L[1..n1+1], R[1..n2+1]
    L[1..n1] = A[p..q],  R[1..n2] = A[q+1..r]
    L[n1+1] = R[n2+1] = INF        // sentinel
    i = j = 1
    for k = p to r:
        if L[i] <= R[j]:
            A[k] = L[i]; i++
        else:
            A[k] = R[j]; j++
```

Phân tích Complexity – Merge Sort

Bước D&C	Chi phí	Giải thích
Divide: tính $q = \lfloor (p+r)/2 \rfloor$	$\Theta(1)$	1 phép chia
Conquer: gọi đệ quy 2 nửa	$2T(n/2)$	2 bài con, mỗi bài $n/2$
Combine: Merge 2 mảng con	$\Theta(n)$	duyệt tuyến tính n phần tử

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$.
Giải (Master Theorem Case 2, $a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_2 2})$): $T(n) = \Theta(n \lg n)$.
Space: Merge tạo mảng tạm L, R tổng $\Theta(n)$ + stack đệ quy $O(\lg n) \to \Theta(n)$.

Recurrence: $T(n) = 2T(n/2) + \Theta(n) \to T(n) = \Theta(n \lg n)$ (Master Theorem Case 2)

2.5.2 Loop Invariant (Merge)

- **Bất biến:** Tại đầu mỗi lần lặp của vòng for trong `Merge`, mảng con output $A[p..k-1]$ chứa $k-p$ phần tử nhỏ nhất của 2 mảng con L và R, đã sắp tăng dần.
- **Init ($k = p$):** $A[p..p-1]$ rỗng – bất biến đúng.
- **Maintenance:** So sánh đầu L và đầu R, chọn phần tử nhỏ hơn đặt vào $A[k]$. Phần tử này $\geq$ tất cả phần tử đã đặt (vì L, R đã sắp) $\to A[p..k]$ vẫn sắp.
- **Termination:** Khi hết cả L lẫn R, $A[p..r]$ chứa tất cả phần tử ban đầu, đã sắp. □

2.5.3 Phân tích Complexity

- **Divide:** Chia đôi $= \Theta(1)$.
- **Conquer:** 2 bài con kích thước $n/2 = 2T(n/2)$.
- **Combine (Merge):** Trộn 2 mảng đã sắp $= \Theta(n)$.
- **Recurrence:** $T(n) = 2T(n/2) + \Theta(n)$. Áp dụng Master Theorem Case 2: $a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_2 2}) \to T(n) = \Theta(n \lg n)$.
- **Space:** $\Theta(n)$ (mảng tạm cho Merge) + $\Theta(\lg n)$ (stack đệ quy) $= \Theta(n)$.
- Worst = Average = Best = $\Theta(n \lg n)$. **Stable**. Không in-place.

2.5.4 Minh họa Animation

Mảng gốc: $[5, 3, 8, 1, 4]$

5	3	8	1	4

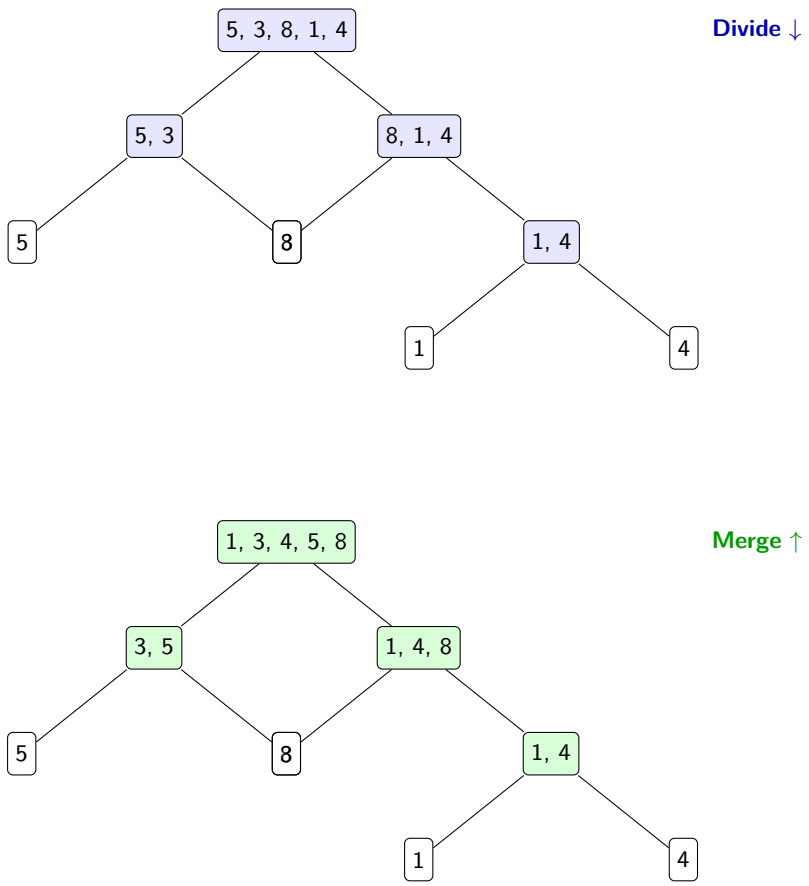


Figure 5: Merge Sort – Chia (Divide) và Trộn (Merge) trên mảng $[5, 3, 8, 1, 4]$

▷ Tự Trace: Insertion Sort – Trace mảng

Cho mảng $A = [5, 3, 8, 1, 4]$. Tự trace Insertion Sort, điền mảng sau mỗi bước:

j	**key**	**Mảng A sau bước**
–	–	$[5, 3, 8, 1, 4]$
2	3	[_____]
3	8	[_____]
4	1	[_____]
5	4	[_____]

2.6 2.6 Quick Sort – CLRS Ch.7

Pseudocode 6: Quick Sort

```
QuickSort(A, p, r):
    if p < r:
        q = Partition(A, p, r)
        QuickSort(A, p, q-1)
        QuickSort(A, q+1, r)

Partition(A, p, r):              // CLRS 7.1
    pivot = A[r]
    i = p - 1
    for j = p to r-1:
        if A[j] <= pivot:
            i = i + 1
            swap(A[i], A[j])
    swap(A[i+1], A[r])
    return i + 1
```

Phân tích Complexity – Quick Sort

Bước	Chi phí	Giải thích
Partition: duyệt $n-1$ phần tử	$\Theta(n)$	1 vòng for
Đệ quy 2 nửa	$T(k)+T(n-k-1)$	k = vị trí pivot

Best case (pivot chia đều): $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \lg n)$.
Worst case (pivot lệch cực): $T(n) = T(n-1) + \Theta(n) \to \Theta(n^2)$.
Average case: Mọi cách chia trung bình cho $\Theta(n \lg n)$.
Space: Stack đệ quy sâu $O(\lg n)$ avg, $O(n)$ worst. Không cần mảng phụ (in-place).

2.6.1 Ý tưởng

Chọn một phần tử làm **pivot** (thường là phần tử cuối). Dùng `Partition` để chia mảng thành 2 phần: bên trái $\leq$ pivot, bên phải $>$ pivot. Pivot nằm đúng vị trí cuối cùng. Đệ quy sắp xếp 2 nửa. **Khác Merge Sort:** công việc nặng nằm ở bước *chia* (Partition), còn merge thì miễn phí (đã in-place). Average $\Theta(n \lg n)$, worst $\Theta(n^2)$ khi pivot luôn lệch cực đoan.

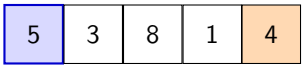
2.6.2 Loop Invariant cho Partition (CLRS §7.1):

Tại đầu mỗi lần lặp for:

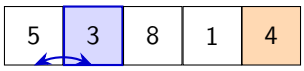
1. $A[p..i]$: tất cả $\leq$ pivot
2. $A[i+1..j-1]$: tất cả $>$ pivot
3. $A[r]$ = pivot

2.6.3 Minh họa Animation

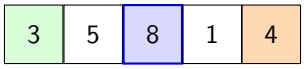
Bước 1 – Partition toàn mảng: $[5, 3, 8, 1, 4]$, Pivot $= A[5] = 4$, i bắt đầu tại 0.



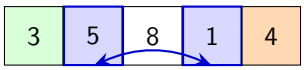
$j = 1$: $5 > 4 \to$ Bỏ qua. $i = 0$



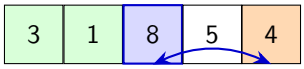
$j = 2$: $3 \leq 4 \to i = 1$, Swap $A[1] \leftrightarrow A[2]$ (5 & 3)



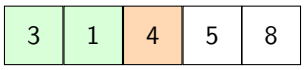
$j = 3$: $8 > 4 \to$ Bỏ qua. $i = 1$



$j = 4$: $1 \leq 4 \to i = 2$, Swap $A[2] \leftrightarrow A[4]$ (5 & 1)

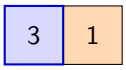


Hết vòng for. Swap $A[i{+}1] \leftrightarrow A[r] = A[3] \leftrightarrow A[5]$ (8 & 4)

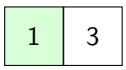


Partition xong: $[3, 1, \boxed{4}, 5, 8]$. Pivot ở $q = 3$.

Bước 2 – Đệ quy trái QuickSort$(A, 1, 2)$ trên $[3, 1]$: Pivot $= A[2] = 1$.

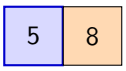


$j = 1$: $3 > 1 \to$ Bỏ qua. $i = 0$. Swap $A[1] \leftrightarrow A[2]$ (3 & 1)

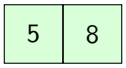


Pivot 1 ở vị trí $q = 1$. Mảng con: $[\boxed{1}, 3]$

Bước 3 – Đệ quy phải QuickSort$(A, 4, 5)$ trên $[5, 8]$: Pivot $= A[5] = 8$.

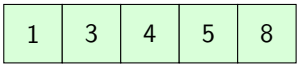


$j = 4$: $5 \leq 8 \to i = 4$, Swap $A[4] \leftrightarrow A[4]$ (không đổi)



Pivot 8 ở vị trí $q = 5$. Mảng con: $[5, \boxed{8}]$ – đã đúng

Kết quả cuối cùng:



Mảng đã được sắp xếp hoàn toàn! $[1, 3, 4, 5, 8]$

Figure 6: Quick Sort – Toàn bộ quá trình sắp xếp trên mảng $[5, 3, 8, 1, 4]$

2.6.4 Phân tích (CLRS §7.2, 7.4):

- **Worst case:** Partition luôn phân hoạch $(0, n-1) \to T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$
- **Best case:** Partition phân hoạch đều $\to T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$
- **Average case (CLRS §7.4.2):** $E[T(n)] = \Theta(n \lg n)$ – chứng minh bằng indicator random variables

2.6.5 Randomized QuickSort (CLRS §7.3):

Pseudocode 7: Randomized Partition

```
RandomizedPartition(A, p, r):
    i = Random(p, r)
    swap(A[i], A[r])
    return Partition(A, p, r)
```

Chọn pivot **ngẫu nhiên** → Expected time = $\Theta(n \lg n)$, tránh worst case có hệ thống.

2.6.6 Phân tích Complexity

- **Best case:** Partition chia đều → $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$.
- **Worst case:** Pivot luôn là min/max (mảng đã sắp + pivot cuối) → $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.
- **Average case:** Phân tích bằng indicator random variables: mọi cách chia đều cho $\Theta(n \lg n)$ trung bình.
- **Space:** $O(\lg n)$ avg (stack đệ quy), $O(n)$ worst case. In-place (không cần mảng phụ). **Không stable**.

✓ **Tự kiểm tra:** Độ phức tạp worst case của Quick Sort là gì? Tại sao trong thực tế ta vẫn dùng Quick Sort thay vì Merge Sort? (Nêu 2 lý do.)

2.7 2.7 Heap Sort – CLRS Ch.6

Trực giác: Heap Sort sử dụng cấu trúc **max-heap** (cây nhị phân hoàn chỉnh lưu trong mảng, gốc luôn lớn nhất). Bước 1: Xây heap $O(n)$. Bước 2: Lần lượt rút gốc (max) ra cuối mảng, rồi vá heap lại. Kết quả: worst case $\Theta(n \lg n)$, in-place, không cần bộ nhớ phụ.

2.7.1 Ý tưởng

Heap Sort khai thác cấu trúc **heap** – cây nhị phân hoàn chỉnh lưu ngay trong mảng – để sắp xếp qua 2 bước:

1. **BuildMaxHeap** ($O(n)$): Biến mảng bất kỳ thành max-heap. Mảng A được coi là cây nhị phân: nút i có con trái $= 2i$, con phải $= 2i + 1$. Gọi `MaxHeapify` từ $\lfloor n/2 \rfloor$ xuống 1 để đảm bảo tính chất cha $\geq$ con tại mọi nút.

2. **Extract-Max lặp lại** ($O(n \lg n)$): Gốc $A[1]$ luôn là phần tử lớn nhất. Swap $A[1] \leftrightarrow A[n]$ (đưa max về cuối), giảm kích thước heap đi 1, rồi gọi `MaxHeapify(1)` để vá lại gốc. Lặp $n - 1$ lần → mảng sắp tăng dần.

Tại sao hay? Kết hợp ưu điểm: worst case $\Theta(n \lg n)$ như Merge Sort *nhưng* in-place $O(1)$ space như Quick Sort.

Tính chất Max-Heap: $A[\lfloor i/2 \rfloor] \geq A[i]$ – cha luôn $\geq$ con.

Tính chất Min-Heap: $A[\lfloor i/2 \rfloor] \leq A[i]$ – cha luôn $\leq$ con. Gốc là phần tử **nhỏ nhất**.

So sánh Min-Heap vs Max-Heap

	Max-Heap	**Min-Heap**
Tính chất	Cha $\geq$ con	Cha $\leq$ con
Gốc (root)	Phần tử **lớn nhất**	Phần tử **nhỏ nhất**
ExtractMax/Min	$O(\lg n)$ – rút max	$O(\lg n)$ – rút min
Insert	$O(\lg n)$	$O(\lg n)$
Build	$O(n)$	$O(n)$
Ứng dụng	**Heap Sort** (rút max ra cuối)	**Dijkstra, Prim** (rút min priority)

Tại sao Heap Sort dùng Max-Heap? Vì ta muốn lần lượt rút phần tử *lớn nhất* ra cuối mảng $\rightarrow$ mảng sắp tăng dần.
Tại sao Dijkstra/Prim dùng Min-Heap? Vì ta muốn `ExtractMin` – luôn xử lý đỉnh có khoảng cách/key *nhỏ nhất* trước (Greedy).

Pseudocode 8: Heap Sort (MaxHeapify + BuildMaxHeap + HeapSort)

```
MaxHeapify(A, i, n):                // CLRS 6.2
    l = 2*i;  r = 2*i + 1
    largest = i
    if l <= n and A[l] > A[largest]: largest = l
    if r <= n and A[r] > A[largest]: largest = r
    if largest != i:
        swap(A[i], A[largest])
        MaxHeapify(A, largest, n)

BuildMaxHeap(A, n):                 // CLRS 6.3
    for i = floor(n/2) downto 1:
        MaxHeapify(A, i, n)

HeapSort(A, n):                     // CLRS 6.4
    BuildMaxHeap(A, n)
    for i = n downto 2:
        swap(A[1], A[i])
        MaxHeapify(A, 1, i-1)
```

Phân tích Complexity – Heap Sort

Thao tác	**Chi phí**	**Giải thích**
`MaxHeapify(A, i)`	$O(\lg n)$	đi xuống tối đa $\lg n$ tầng
`BuildMaxHeap(A)`	$O(n)$	*không phải* $O(n \lg n)$!
(Lý do: $\sum_{h=0}^{\lg n} \lceil n/2^{h+1} \rceil \cdot O(h) = O(n)$)		tổng telescoping
`HeapSort`: vòng for $n-1$ lần	$(n-1) \cdot O(\lg n)$	swap gốc + MaxHeapify

Time: BuildMaxHeap $O(n)$ + HeapSort loop $O(n \lg n) \rightarrow \Theta(n \lg n)$ mọi trường hợp.
Space: $O(1)$ – heap lưu ngay trong mảng A (in-place), không cần mảng phụ.

2.7.2 Complexity

BuildMaxHeap $= O(n)$. HeapSort $= \Theta(n \lg n)$ (worst, average, best). Space $= O(1)$ (in-place). Không stable.

2.7.3 Loop Invariant

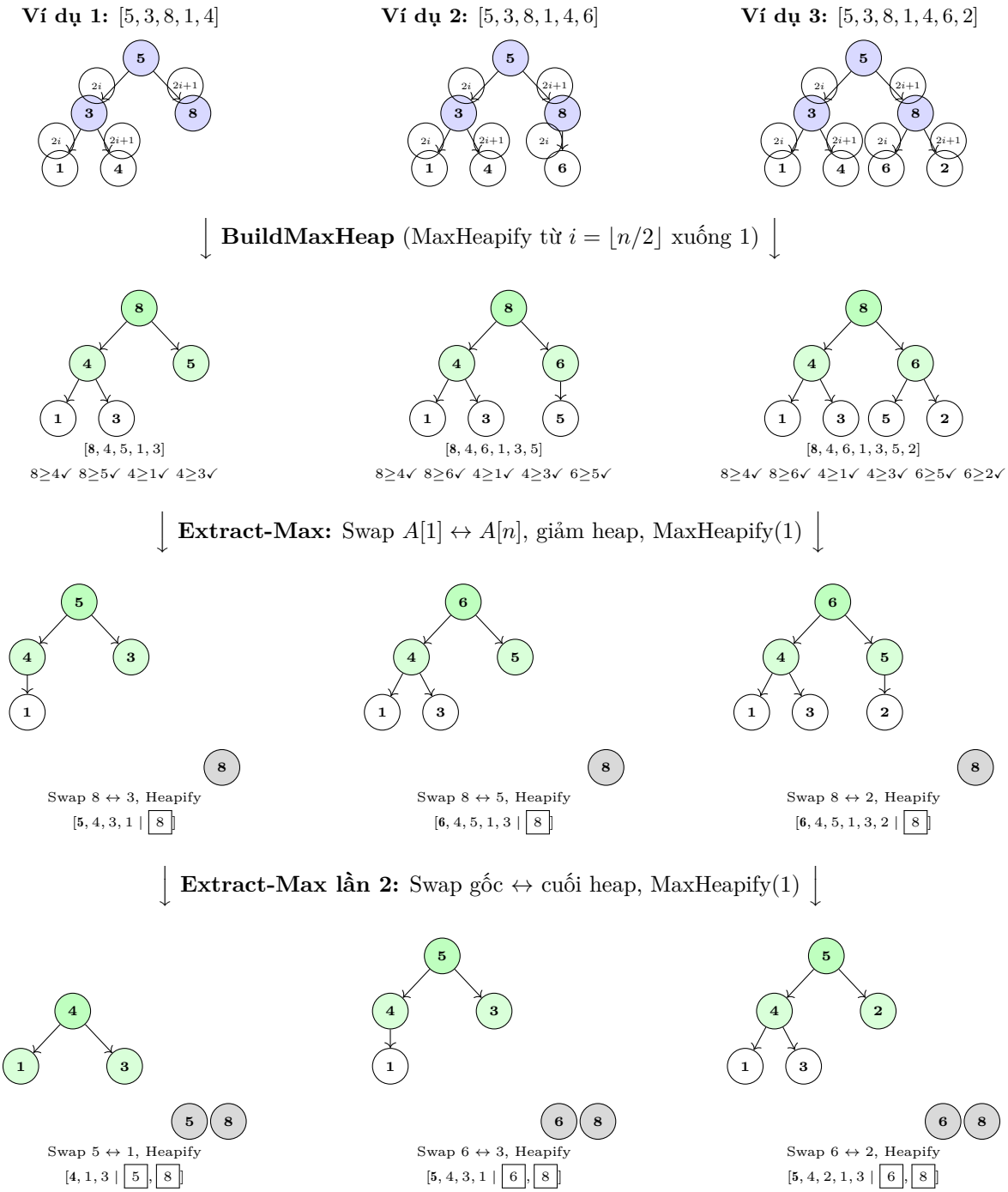
- **Bất biến:** Tại đầu mỗi lần lặp `for i = n downto 2`: (1) $A[1..i]$ là max-heap, và (2) $A[i{+}1..n]$ chứa $n{-}i$ phần tử lớn nhất của mảng, đã sắp tăng dần.
- **Init ($i = n$):** BuildMaxHeap vừa chạy xong $\rightarrow$ $A[1..n]$ là max-heap. $A[n{+}1..n]$ rỗng – bất biến đúng.

- **Maintenance:** $A[1]$ = max của heap $\rightarrow$ swap $A[1] \leftrightarrow A[i] \rightarrow A[i]$ đúng chỗ. Gọi MaxHeapify$(A, 1, i-1)$ để sửa heap. Bất biến đúng cho $i-1$.
- **Termination ($i = 1$):** $A[2..n]$ chứa $n-1$ phần tử lớn nhất, sắp. $A[1]$ là nhỏ nhất $\rightarrow$ mảng sắp. $\square$

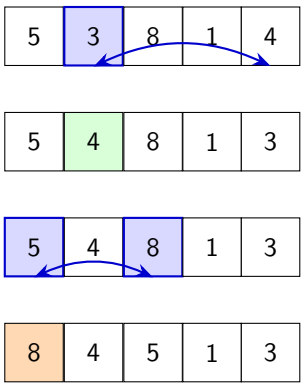
2.7.4 Minh họa Animation

Cây nhị phân (Binary Tree) biểu diễn Heap:

Mảng $A = [5, 3, 8, 1, 4]$ được lưu trong heap theo quy tắc: nút i có con trái $= 2i$, con phải $= 2i + 1$.



Bước 1 – BuildMaxHeap: $[5, 3, 8, 1, 4]$



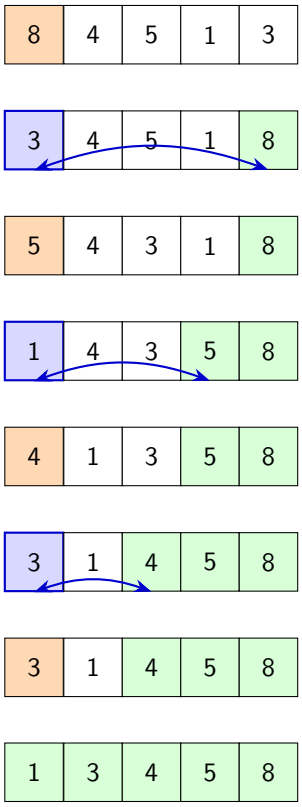
$i = 2$: $A[2]$=3, con $A[5]$=4 $> 3 \rightarrow$ Swap

Kết quả: $[5, 4, 8, 1, 3]$

$i = 1$: $A[1]$=5, con $A[3]$=8 $> 5 \rightarrow$ Swap

Max-Heap: $[\mathbf{8}, 4, 5, 1, 3]$

Bước 2 – Extract-Max lần lượt:



Heap: $[8, 4, 5, 1, 3]$

Swap $A[1] \leftrightarrow A[5]$. Heapify $\rightarrow$

$[\mathbf{5}, 4, 3, 1 \mid 8]$

Swap $A[1] \leftrightarrow A[4]$. Heapify $\rightarrow$

$[\mathbf{4}, 1, 3 \mid 5, 8]$

Swap $A[1] \leftrightarrow A[3]$. Heapify $\rightarrow$

$[\mathbf{3}, 1 \mid 4, 5, 8]$. Swap $A[1] \leftrightarrow A[2]$:

Mảng đã sắp xếp: $[1, 3, 4, 5, 8]$

Figure 7: Heap Sort – Toàn bộ quá trình sắp xếp trên mảng $[5, 3, 8, 1, 4]$

2.8 Cận dưới cho sắp xếp so sánh – CLRS §8.1

Định lý (CLRS Theorem 8.1): Mọi thuật toán sắp xếp dựa trên so sánh cần $\Omega(n \lg n)$ phép so sánh trong worst case.

Chứng minh: Cây quyết định (decision tree) có $n!$ lá $\rightarrow$ chiều cao $\geq \lg(n!) = \Omega(n \lg n)$.

$\rightarrow$ **Merge Sort và Quick Sort (average) đều TỐI ƯU** cho comparison sorting!

2.9 2.8 Counting Sort – CLRS Ch.8

2.9.1 Ý tưởng

Thay vì so sánh, ta **đếm** số lần xuất hiện của mỗi giá trị. Tạo mảng đếm $C[0..k]$, với $C[x]$ = số phần tử $= x$. Cộng dồn C để biết vị trí cuối cùng của mỗi giá trị trong output. Duyệt ngược mảng gốc, đặt từng phần tử vào đúng vị trí → **stable**, $O(n+k)$. Chỉ dùng khi biết phạm vi giá trị k và k không quá lớn.

Pseudocode 9: Counting Sort (CLRS 8.2)

```
CountingSort(A, B, n, k):
    // A: input, B: output, k: max value
    let C[0..k] = all zeros
    for j = 1 to n:
        C[A[j]] = C[A[j]] + 1      // dem tan suat
    for i = 1 to k:
        C[i] = C[i] + C[i-1]       // cong don -> vi tri
    for j = n downto 1:             // duyet NGUOC -> stable
        B[C[A[j]]] = A[j]
        C[A[j]] = C[A[j]] - 1
```

Phân tích Complexity – Counting Sort

Dòng code	Số lần chạy	Chi phí
`let C[0..k] = all zeros`	$k+1$	khởi tạo mảng đếm
`for j = 1 to n: C[A[j]]++`	n	đếm tần suất
`for i = 1 to k: C[i] += C[i-1]`	k	cộng dồn
`for j = n downto 1: B[C[A[j]]]=A[j]`	n	đặt vào output

Time: $\Theta(n+k)$ – 2 vòng $\Theta(n)$ + 1 vòng $\Theta(k)$. *Tuyến tính* khi $k = O(n)$.
Space: Mảng $C[0..k]$: $\Theta(k)$ + mảng output $B[1..n]$: $\Theta(n) \to \Theta(n+k)$. *Không* in-place.

2.9.2 Complexity

$\Theta(n+k)$ time, $\Theta(n+k)$ space. Stable. **Không dựa trên so sánh** – vì vậy không bị ràng buộc bởi cận dưới $\Omega(n \lg n)$.

? Tại sao? Tại sao **Counting Sort** không vi phạm cận dưới $\Omega(n \lg n)$, dù nó chạy $O(n+k)$? Gợi ý: xem lại điều kiện "sắp xếp dựa trên so sánh" trong Theorem 8.1 (CLRS).

---

3 Chương 3: Định lý Thợ và Giải đệ quy – Master Theorem & Solving Recurrences (CLRS Ch.4)

Trực giác: Hầu hết thuật toán Divide & Conquer đều có công thức đệ quy dạng $T(n) = aT(n/b) + f(n)$. Master Theorem là **công thức cookbook** giải ngay đa số trường hợp. Nếu không áp dụng được, dùng Recursion Tree để vẽ trực quan tổng chi phí theo từng tầng đệ quy.

Sai lầm thường gặp: (1) Master Theorem không áp dụng được khi $b < 1$ hoặc $a < 1$, hoặc khi $f(n)$ không đơn điệu. (2) Case 3 đòi hỏi điều kiện **regularity** $af(n/b) \le cf(n)$ – nhiều người quên kiểm tra. (3) $T(n) = 2T(n/2) + n \lg n$ là Case 2 (không phải Case 3) vì $f(n) = n \lg n = \Theta(n^{\log_2 2} \cdot \lg n)$.

3.1 Ý tưởng Chia để trị (Divide & Conquer)

Divide & Conquer là mô hình thiết kế thuật toán gồm **3 bước**:

1. **Divide (Chia):** Chia bài toán kích thước n thành a bài toán con kích thước n/b.
2. **Conquer (Trị):** Giải đệ quy từng bài toán con. Nếu bài toán con đủ nhỏ, giải trực tiếp (base case).
3. **Combine (Kết hợp):** Gộp nghiệm các bài toán con thành nghiệm bài toán gốc.

Công thức đệ quy tổng quát:

$$T(n) = aT(n/b) + f(n)$$

trong đó a = số bài toán con, n/b = kích thước mỗi bài con, $f(n)$ = chi phí chia + gộp.

Ví dụ điển hình:

- **Merge Sort:** $a = 2, b = 2, f(n) = \Theta(n)$ (chia giữa, merge $\Theta(n)$) $\to T(n) = \Theta(n \lg n)$.
- **Quick Sort:** $a = 2, b$ phụ thuộc pivot, $f(n) = \Theta(n)$ (Partition).
- **Binary Search:** $a = 1, b = 2, f(n) = \Theta(1) \to T(n) = \Theta(\lg n)$.

3.2 Ba phương pháp giải đệ quy (CLRS §4.3-4.5)

3.2.1 Phương pháp 1: Substitution Method (Thay thế) – CLRS §4.3

1. **Đoán** dạng nghiệm
2. **Chứng minh** bằng quy nạp toán học

Ví dụ: $T(n) = 2T(\lfloor n/2 \rfloor) + n$, đoán $T(n) = O(n \lg n)$.

- Giả sử $T(\lfloor n/2 \rfloor) \le c \cdot \lfloor n/2 \rfloor \cdot \lg(\lfloor n/2 \rfloor)$
- $T(n) \le 2c(n/2)\lg(n/2) + n = cn \lg n - cn \lg 2 + n = cn \lg n - cn + n$
- $\le cn \lg n$ khi $c \ge 1$ (OK)

3.2.2 Phương pháp 2: Recursion Tree (Cây đệ quy) – CLRS §4.4

Vẽ cây, mỗi nút là chi phí tại mức đệ quy đó.

Ví dụ: $T(n) = 3T(n/4) + cn^2$

Pseudocode 9: Cây đệ quy (Recursion Tree)

```
Muc 0:  cn^2                                -> cn^2
Muc 1:  3 * c(n/4)^2                        -> (3/16) cn^2
Muc 2:  9 * c(n/16)^2                       -> (3/16)^2 cn^2
...
Muc i:  3^i * c(n/4^i)^2                    -> (3/16)^i cn^2
...
Muc log_4(n): 3^(log_4 n) nut, Theta(1)   -> Theta(n^(log_4 3))

Tong = cn^2 * Sum (3/16)^i (i=0..inf) = cn^2 * 16/13 = O(n^2)
```

3.2.3 Phương pháp 3: Master Theorem – CLRS §4.5

Cho $T(n) = aT(n/b) + f(n)$**,** $a \ge 1, b > 1$:

Case	Điều kiện	Kết quả
1	$f(n) = O(n^{\log_b a - \varepsilon})$ với $\varepsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \cdot \lg^k n)$	$T(n) = \Theta(n^{\log_b a} \cdot \lg^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ với $\varepsilon > 0$ **VÀ** $af(n/b) \le cf(n)$ với $c < 1$	$T(n) = \Theta(f(n))$

Table 4: Master Theorem – Ba trường hợp

3.2.4 Ví dụ áp dụng:

Đệ quy	a	b	$\log_b a$	$f(n)$	Case	Kết quả
$T(n) = 2T(n/2) + n$	2	2	1	n	2 ($k = 0$)	$\Theta(n \lg n)$
$T(n) = T(n/2) + 1$	1	2	0	1	2 ($k = 0$)	$\Theta(\lg n)$
$T(n) = 7T(n/2) + n^2$	7	2	2.807	n^2	1 ($\varepsilon \approx 0.8$)	$\Theta(n^{2.807})$
$T(n) = 9T(n/3) + n$	9	3	2	n	1 ($\varepsilon = 1$)	$\Theta(n^2)$
$T(n) = T(2n/3) + 1$	1	3/2	0	1	2 ($k = 0$)	$\Theta(\lg n)$

Table 5: Ví dụ áp dụng Master Theorem

---

4 Chương 4: Lý thuyết Đồ thị – Graph Theory (CLRS Ch.22)

Trực giác: Đồ thị là cấu trúc dữ liệu mô hình hóa mọi thứ: mạng lưới đường đi, kết nối xã hội, phụ thuộc công việc... Hiểu **hai cách biểu diễn** (danh sách kề vs ma trận kề) là bước đầu tiên, vì lựa chọn biểu diễn ảnh hưởng trực tiếp tới độ phức tạp của thuật toán trên đó.

Sai lầm thường gặp: (1) Nhầm giữa đồ thị có hướng và vô hướng – Dijkstra chạy trên đồ thị có hướng. (2) Ma trận kề tốn $\Theta(V^2)$ bộ nhớ – không hợp lý cho đồ thị thưa (mạng xã hội hàng triệu node). (3) Đồ thị không nhất thiết liên thông – luôn kiểm tra điều kiện liên thông trước khi chạy MST.

4.1 Định nghĩa (CLRS §B.4, §22.1)

- **Đồ thị** $G = (V, E)$**:** V = tập đỉnh, $E \subseteq V \times V$ = tập cạnh
- **Vô hướng:** $(u, v) \in E \Leftrightarrow (v, u) \in E$
- **Có hướng:** $(u, v) \in E$ không suy ra $(v, u) \in E$
- **Bậc** $\deg(v)$**:** Số cạnh kề v. Đồ thị có hướng: in-degree + out-degree
- **Đường đi:** Dãy $\langle v_0, v_1, \ldots, v_k \rangle$ sao cho $(v_i, v_{i+1}) \in E$
- **Chu trình:** Đường đi với $v_0 = v_k$, $k \geq 1$

Handshaking Lemma: $\sum \deg(v) = 2|E|$ (vô hướng)

? Tại sao? Ma trận kề tốn $\Theta(V^2)$ bộ nhớ kể cả khi đồ thị rất thưa ($E \ll V^2$). Tại sao? Và tại sao danh sách kề lại chỉ tốn $\Theta(V + E)$? Hãy vẽ ví dụ cụ thể với 5 đỉnh, 3 cạnh.

⇒ Giải thích bằng lời bạn: Giải thích Handshaking Lemma bằng ngôn ngữ đời thường. Gợi ý: nghĩ về "bắt tay" – nếu 5 người bắt tay nhau, tổng số "lần mà ai đó bắt tay" liên quan thế nào đến "số cái bắt tay"?

4.2 Biểu diễn (CLRS §22.1)

4.2.1 1) Danh sách kề – Space: $\Theta(V + E)$

Pseudocode 10: Danh sách kề (Adjacency List)

```
Vi du: V = {a,b,c,d,e}
  a -> [b, d]
  b -> [a, d, e]
  c -> [d, e]
  d -> [a, b]
  e -> [b, c]
```

Tốt cho đồ thị **thưa** (sparse: $E \ll V^2$)

4.2.2 2) Ma trận kề – Space: $\Theta(V^2)$

Pseudocode 11: Ma trận kề (Adjacency Matrix)

```
A[i][j] = 1 neu (i,j) in E, = 0 nguoc lai

    a b c d e
a [ 0 1 0 1 0 ]
b [ 1 0 0 1 1 ]
c [ 0 0 0 1 1 ]
d [ 1 1 0 0 0 ]
e [ 0 1 1 0 0 ]
```

Tốt cho đồ thị **dày** (dense: $E \approx V^2$). Kiểm tra cạnh $O(1)$.

Tính chất (CLRS): $M = A^p$ thì $M[i][j]$ = **số đường đi dài** p từ i đến j.

4.3 Cây (Tree) – CLRS §B.5

Cây = đồ thị vô hướng liên thông, không chu trình

Định lý tương đương (CLRS Theorem B.2): Cho $G = (V, E)$ vô hướng, các điều kiện tương đương:

1. G là cây (liên thông + không chu trình)
2. G liên thông và $|E| = |V| - 1 \to$ **Prim dùng tính chất này**
3. G không chu trình và $|E| = |V| - 1 \to$ **Kruskal dùng tính chất này**
4. Giữa mọi 2 đỉnh có **đúng 1** đường đi

4.4 4.4 Đồ thị có trọng số

$G = (V, E, \ell)$, $\ell : E \to \mathbb{R}^+$. Nếu $ij \in E$ thì $\ell(ij)$ = trọng số cạnh.

---

5 Chương 5: Thuật toán Tham lam và Cây Bao trùm Nhỏ nhất – Greedy Algorithms & Minimum Spanning Tree (CLRS Ch.16, 23)

Trực giác: Greedy = **tham lam**: mỗi bước chỉ nghĩ đến lợi ích tức thời, không nhìn xa. Điều kỳ diệu là với MST, chiến lược "tham lam" lại cho nghiệm tối ưu toàn cục – nhờ tính chất Cut. Hãy tưởng tượng xây mạng điện tiết kiệm nhất: ta chỉ cần liên tục chọn cạnh rẻ nhất chưa tạo vòng lặp (Kruskal) hoặc cạnh rẻ nhất nối ra ngoài cụm (Prim).

Sai lầm thường gặp: (1) Greedy **KHÔNG phải** luôn đúng – phải chứng minh 2 tính chất (Greedy-choice + Optimal substructure). Ví dụ: Greedy không cho MST tối ưu nếu ta tham lam chọn cạnh ngắn nhất theo đỉnh thay vì theo cạnh. (2) Trong Kruskal, kiểm tra chu trình phải dùng Union-Find $O(\alpha(V))$ – không dùng DFS $O(V)$ vì sẽ làm tổng trở thành $O(EV)$.

5.1 Ý tưởng thuật toán Tham lam (Greedy)

Thuật toán Tham lam xây dựng nghiệm từng bước, mỗi bước chọn phương án **tốt nhất tại thời điểm đó** (locally optimal) mà không quay lui. Khác với DP (xét mọi khả năng), Greedy chỉ xét **1 lựa chọn duy nhất** mỗi bước.

Khi nào Greedy cho nghiệm tối ưu? Khi bài toán thỏa mãn 2 tính chất:

1. **Greedy-choice property:** Tồn tại nghiệm tối ưu toàn cục chứa lựa chọn tham lam (chọn cục bộ tốt nhất không loại trừ nghiệm tối ưu).
2. **Optimal substructure:** Sau khi chọn greedy, bài toán con còn lại cũng có nghiệm tối ưu.

So sánh với DP: Greedy quyết định *trước khi* giải bài toán con (top-down, không quay lui). DP quyết định *sau khi* giải hết bài toán con (bottom-up hoặc memoized). Greedy nhanh hơn ($O(n \lg n)$ vs $O(n^2)$ hoặc $O(n^3)$) nhưng chỉ đúng khi có 2 tính chất trên.

5.2 5.1 Phương pháp Greedy (CLRS Ch.16)

5.2.1 Hai tính chất cần kiểm tra (CLRS §16.2):

(1) Greedy-choice property: Lựa chọn tham lam tại mỗi bước **không loại trừ** lời giải tối ưu toàn cục. (Tồn tại nghiệm tối ưu chứa lựa chọn greedy)

(2) Optimal substructure: Sau khi chọn greedy, bài toán con còn lại cũng có cấu trúc tối ưu.

5.2.2 Framework Greedy (từ bài giảng):

Pseudocode 12: Greedy Framework

```
Input: Tap S, dieu kien (1), ham muc tieu f
Buoc 0: T := empty
Lap: Chon a in S \ T sao cho:
    - a thoa dieu kien (2)
    - a toi uu CUC BO tai buoc nay
Dung khi dieu kien (3).
```

Phân tích Complexity – Greedy Framework (tổng quát)

Bước	Chi phí	Giải thích
`T := empty`	$\Theta(1)$	khởi tạo
`Lặp: Chọn a tối ưu cục bộ`	tùy cách chọn	xem thuật toán cụ thể

Time: Phụ thuộc bài toán cụ thể. Thường gồm: **sắp xếp** $O(n \lg n)$ + **duyệt** $O(n) \to O(n \lg n)$.
Space: $O(n)$ cho tập T kết quả. Cấu trúc dữ liệu phụ tùy bài.

> **CLRS nhấn mạnh:** Greedy KHÔNG phải lúc nào cũng đúng! Phải **chứng minh** 2 tính chất trên.

5.3 5.2 MST – Cây bao trùm nhỏ nhất (CLRS Ch.23)

Bài toán: Cho $G = (V, E, w)$ vô hướng, liên thông, có trọng số. Tìm cây $T = (V, F)$ sao cho $w(T) = \sum w(e)$ nhỏ nhất.

5.3.1 Định lý Cut (CLRS Theorem 23.1) – NỀN TẢNG cho cả Kruskal và Prim

Cho: Cut $(S, V - S)$ của G. Cạnh **nhẹ nhất** (light edge) qua cut mà **an toàn** (safe) thì thuộc MST.

Chính xác hơn: Cho $A \subseteq$ MST nào đó. Cut $(S, V - S)$ **respect** A (không cạnh nào của A qua cut). Cạnh nhẹ nhất qua cut là **safe edge** cho $A \to$ có thể thêm vào A mà A vẫn $\subseteq$ MST.

5.3.2 Kruskal (CLRS §23.2)

Ý tưởng: Xét cạnh theo trọng số tăng dần, thêm nếu không tạo chu trình.

Pseudocode 13: Kruskal's Algorithm

```
Kruskal(G, w):
    A = empty
    for moi dinh v in V:
        MakeSet(v)
    Sap xep E theo w tang dan
    for moi canh (u,v) in E (theo thu tu w tang):
        if FindSet(u) != FindSet(v):
            A = A + {(u,v)}
            Union(u, v)
    return A
```

Phân tích Complexity – Kruskal's Algorithm

Dòng code	Số lần chạy	Chi phí
`for v in V: MakeSet(v)`	V lần	$O(V)$
`Sắp xếp E theo w tăng dần`	1 lần	$O(E \lg E)$
`for (u,v) in E:`	E lần	duyệt tất cả cạnh
`  FindSet(u), FindSet(v)`	$2E$ lần	$O(\alpha(V))$ mỗi lần
`  Union(u,v)`	tối đa $V - 1$	$O(\alpha(V))$ mỗi lần

Time: Sắp xếp $O(E \lg E)$ + duyệt E cạnh × Union-Find $O(\alpha(V)) \to$ **tổng** $O(E \lg E) = O(E \lg V)$. (Vì $E \leq V^2 \Rightarrow \lg E \leq 2 \lg V = O(\lg V)$. $\alpha(V)$ gần như hằng số.)
Space: Mảng `parent[V]` + `rank[V]` cho Union-Find $\to O(V)$.

Chứng minh đúng (CLRS): Khi thêm (u, v), cut (component chứa u, phần còn lại) được respect bởi A, và (u, v) là cạnh nhẹ nhất qua cut này $\to$ safe edge $\to$ đúng theo Theorem 23.1. (OK)

Complexity: $O(E \lg E) = O(E \lg V)$ – do sắp xếp cạnh. Union-Find: $O(\alpha(V))$ amortized.

Minh họa trực quan Kruskal (Visual Trace – đồ thị giống video YouTube):

Video hướng dẫn: Kruskal's Algorithm – YouTube (click để xem)

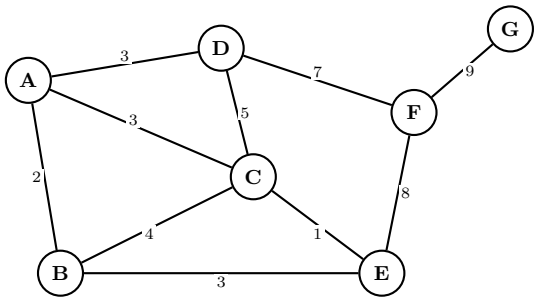


Figure 8: (a) Đồ thị gốc G – 10 cạnh

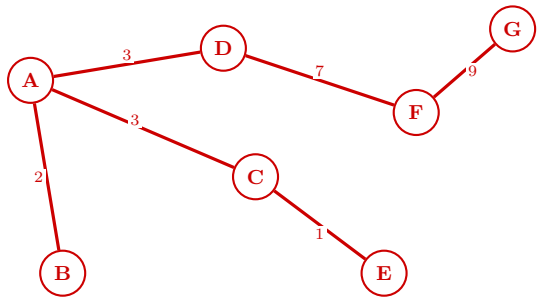


Figure 9: (b) MST kết quả – 6 cạnh, tổng = 25

Cho đồ thị G với $V = \{A, B, C, D, E, F, G\}$ và các cạnh:

Cạnh	CE	AB	AC	AD	BE	BC	CD	DF	EF	FG
w	1	2	3	3	3	4	5	7	8	9

(Lưu ý: đồ thị Kruskal video không có cạnh CF – khác với đồ thị Prim video bên dưới.)

Sắp xếp cạnh theo w tăng dần: **CE(1), AB(2), AC(3), AD(3), BE(3), BC(4), CD(5), DF(7), EF(8), FG(9)**.

Ban đầu: $\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}, \{G\}$. Cần $|V| - 1 = 6$ cạnh.

Bước	**Cạnh (w)**	**Trạng thái Disjoint Sets**	**Cạnh → MST?**
1	**CE** (1)	$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}, \{G\}$ C và E khác tập → **Union(C,E)**	**Thêm vào MST**
2	**AB** (2)	$\{A\}, \{B\}, \{C, E\}, \{D\}, \{F\}, \{G\}$ A và B khác tập → **Union(A,B)**	**Thêm vào MST**
3	**AC** (3)	$\{A, B\}, \{C, E\}, \{D\}, \{F\}, \{G\}$ $A \in \{A, B\}$, $C \in \{C, E\}$ khác → **Union(A,C)**	**Thêm vào MST**
4	**AD** (3)	$\{A, B, C, E\}, \{D\}, \{F\}, \{G\}$ $A \in \{A, B, C, E\}$, $D \in \{D\}$ khác → **Union(A,D)**	**Thêm vào MST**
5	**BE** (3)	$\{A, B, C, D, E\}, \{F\}, \{G\}$ B và E cùng thuộc $\{A, B, C, D, E\}$ → Chu trình!	**Bỏ (chu trình)**
6	**BC** (4)	$\{A, B, C, D, E\}, \{F\}, \{G\}$ Cùng tập → Chu trình!	**Bỏ (chu trình)**
7	**CD** (5)	$\{A, B, C, D, E\}, \{F\}, \{G\}$ Cùng tập → Chu trình!	**Bỏ (chu trình)**
8	**DF** (7)	$\{A, B, C, D, E\}, \{F\}, \{G\}$ $D \in \{A, B, C, D, E\}$, $F \in \{F\}$ khác → **Union(D,F)**	**Thêm vào MST**
9	**EF** (8)	$\{A, B, C, D, E, F\}, \{G\}$ Cùng tập → Chu trình!	**Bỏ (chu trình)**
10	**FG** (9)	$\{A, B, C, D, E, F\}, \{G\}$ F và G khác tập → **Union(F,G)**	**Thêm vào MST**
$\{A, B, C, D, E, F, G\}$ – $\lvert F\rvert = 6 = \lvert V\rvert - 1$ → **DỪNG**.			

Table 6: Kruskal – Trace từng bước trên đồ thị mẫu

MST: $\{CE, AB, AC, AD, DF, FG\}$, tổng $= 1 + 2 + 3 + 3 + 7 + 9 = \boxed{25}$.

5.3.3 Prim (CLRS §23.2)

Ý tưởng: Mở rộng cây từ đỉnh s, mỗi bước thêm cạnh nhẹ nhất nối cây ra ngoài.

Pseudocode 14: Prim's Algorithm

```
Prim(G, w, s):
    for moi u in V:
        key[u] = INF
        pi[u] = NIL
    key[s] = 0
    Q = MinPriorityQueue(V, key)
    while Q != empty:
        u = ExtractMin(Q)
        for moi v in Adj[u]:
            if v in Q and w(u,v) < key[v]:
                pi[v] = u
                key[v] = w(u,v)        // DecreaseKey
    return {(v, pi[v]) : v in V - {s}}
```

Phân tích Complexity – Prim's Algorithm

Dòng code	Số lần chạy	Chi phí
`for u in V: key[u]=INF`	V	$O(V)$ khởi tạo
`Q = MinPriorityQueue(V)`	1	$O(V)$ build heap
`u = ExtractMin(Q)`	V lần	$O(\lg V)$ mỗi lần
`for v in Adj[u]:`	tổng $= 2E$	mỗi cạnh xét 2 lần
`DecreaseKey(v)`	tối đa E lần	$O(\lg V)$ mỗi lần

Time (Min-Heap): $V \cdot O(\lg V)$ ExtractMin $+ E \cdot O(\lg V)$ DecreaseKey $\to O((V+E)\lg V) = O(E \lg V)$.
(Nếu dùng Fibonacci Heap: DecreaseKey $O(1)$ amortized $\to O(E + V \lg V)$.)
Space: Mảng `key[V]`, π`[V]`, Priority Queue Q chứa V đỉnh $\to O(V)$.

Chứng minh đúng (CLRS): Cut (cây đang xây, đỉnh chưa thêm) luôn được respect. Cạnh (u, v) chọn là nhẹ nhất qua cut $\to$ safe edge. (OK)

Complexity: $O(E \lg V)$ dùng binary heap. $O(E + V \lg V)$ dùng Fibonacci heap.

Minh họa trực quan Prim (Visual Trace, bắt đầu từ A, đồ thị 7 đỉnh – giống YouTube video):

Video hướng dẫn: Prim's Algorithm – YouTube (click để xem)

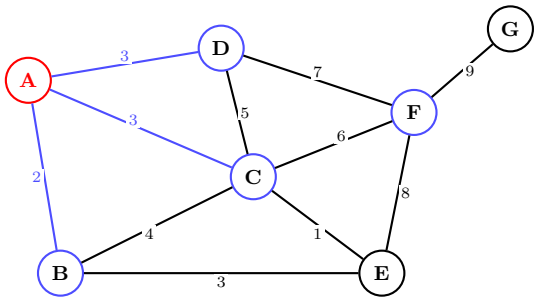


Figure 10: (a) Đồ thị gốc G – 11 cạnh. Visited=$\{A\}$

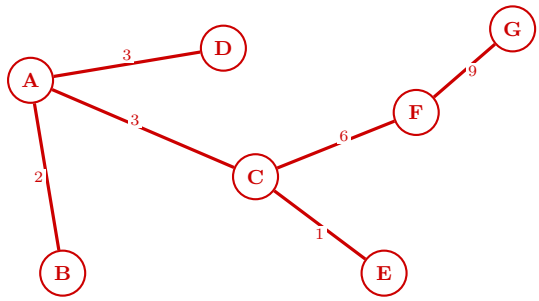


Figure 11: (b) MST kết quả – 6 cạnh, tổng = 24

Cho đồ thị G với $V = \{A, B, C, D, E, F, G\}$ và các cạnh:

Cạnh	CE	AB	AC	AD	BE	BC	CD	CF	DF	EF	FG
w	1	2	3	3	3	4	5	6	7	8	9

Ý tưởng then chốt: duy trì tập Visited (S) và Unvisited. Tại mỗi bước, tìm **cạnh bắt ngang** nhỏ nhất nối $S \to V \setminus S$ rồi thêm đỉnh mới vào S.

Bước	Visited (S)	Cạnh bắt ngang (Crossing Edges)	Chọn (Min)
0	$\{A\}$	**AB**(2), AC(3), AD(3).	**AB** (2)
1	$\{A, B\}$	**AC**(3), AD(3), BE(3), BC(4). *(AB nằm trong $S \to$ bỏ)*	**AC** (3)
2	$\{A, B, C\}$	**CE**(1), AD(3), BE(3), BC(4), CD(5), CF(6). *(C mới đem vào cạnh: CE(1), CD(5), CF(6))*	**CE** (1)
3	$\{A, B, C, E\}$	**AD**(3), BE(3), CD(5), CF(6), EF(8). *BC, AC trong $S \to$ loại. BE: B trong S, E trong $S \to$ loại*	**AD** (3)
4	$\{A, B, C, D, E\}$	**CF**(6), DF(7), EF(8). *CD, AD trong $S \to$ loại. Thêm DF(7) từ D*	**CF** (6)
5	$\{A, B, C, D, E, F\}$	**FG**(9). *(DF, EF nằm trong $S \to$ loại)*	**FG** (9)
6	$\{A, B, C, D, E, F, G\}$	$\lvert S\rvert = 7 = \lvert V\rvert \to$ **DỪNG**.	–

Table 7: Prim – Trace từng bước trên đồ thị mẫu

MST: $\{AB, AC, CE, AD, CF, FG\}$, tổng $= 2+3+1+3+6+9 = \boxed{24}$. *(Lưu ý: đồ thị Prim video có thêm cạnh CF(6) so với Kruskal video, nên MST khác: Prim chọn CF(6) thay vì DF(7), tổng 24 thay vì 25.)*

5.3.4 So sánh trực diện Kruskal vs Prim

Tiêu chí	Kruskal	Prim
Chiến lược	Duyệt **cạnh** nhỏ nhất **toàn cục**, thêm nếu không tạo chu trình	Mở rộng **cây** từ 1 đỉnh, chọn cạnh nhẹ nhất **nối ra ngoài cây**
Cấu trúc dữ liệu	**Union-Find** (Disjoint Sets)	**Min-Priority Queue**
Bước đầu tiên	Sắp xếp tất cả cạnh theo w	Chọn đỉnh bắt đầu s
Kiểm tra chu trình	FindSet(u) $\neq$ FindSet(v)?	Đỉnh v chưa trong cây? ($v \in Q$?)
Complexity	$O(E \lg V)$ (sort + Union-Find)	$O(E \lg V)$ (binary heap) $O(E + V \lg V)$ (Fibonacci heap)
Phù hợp khi	Đồ thị **thưa** ($E \approx V$)	Đồ thị **dày** ($E \approx V^2$)
Cần sắp xếp cạnh?	**Có** (toàn bộ E cạnh)	**Không**
Chứng minh đúng	Cut Theorem: cạnh nhẹ nhất qua cut giữa 2 component	Cut Theorem: cạnh nhẹ nhất qua cut (cây, phần còn lại)
MST forests	Xây nhiều cây rồi hợp nhất	Luôn xây **1 cây duy nhất**

Table 8: So sánh Kruskal và Prim – Đặc điểm và Ưu điểm

▷ Tự Trace: Kruskal – Trace từ đầu

Cho đồ thị 4 đỉnh $\{1, 2, 3, 4\}$ với cạnh: $(1, 2, w{=}4)$, $(1, 3, w{=}1)$, $(2, 3, w{=}3)$, $(2, 4, w{=}2)$, $(3, 4, w{=}5)$.
Sắp xếp cạnh tăng dần: _____

Bước	Cạnh	w	Chu trình?	Kết quả
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____

Tổng trọng số MST = _____

? Tại sao? Tại sao cạnh nhẹ nhất qua một cut **phải** thuộc MST? Thử dùng phản chứng: giả sử cạnh nhẹ nhất (u, v) qua cut KHÔNG thuộc MST T^*. Thêm (u, v) vào T^* tạo ra gì? Điều gì xảy ra?

---

6 Chương 6: Đường đi Ngắn nhất – Shortest Paths: Dijkstra (CLRS Ch.24)

Trực giác: GPS tìm đường đi ngắn nhất chính là Dijkstra (hoặc A*). Ý tưởng: luôn **xử lý đỉnh gần nhất trước** (Greedy), rồi cập nhật (relax) các hàng xóm. Mỗi đỉnh khi được "khóa" vào tập S sẽ không bao giờ bị cập nhật lại – đây là bất biến cốt lõi của Dijkstra. Kết hợp tư duy Greedy (chọn min) và DP (relaxation = cập nhật bảng).

Sai lầm thường gặp: (1) **Dijkstra KHÔNG hoạt động với cạnh âm** – phải dùng Bellman-Ford. (2) Khởi tạo $d[s] = 0$, $d[v] = \infty$ (không phải $-\infty$ hay 0). (3) Khi có nhiều đỉnh cùng khoảng cách minimun, chọn bất kỳ đều đúng – không ảnh hưởng kết quả. (4) Dijkstra cho **single-source** – nếu cần all-pairs hãy dùng Floyd-Warshall.

6.1 Bài toán (CLRS §24.0)

Cho $G = (V, E, w)$ có hướng, $w : E \to \mathbb{R}^+$. Đỉnh nguồn s.

3 biến thể:

1. **Single-pair:** $s \to t$
2. **Single-source:** $s \to$ tất cả v – **(Dijkstra)**
3. **All-pairs:** mọi $u \to v$ – **(Floyd-Warshall)**

Tính chất cấu trúc tối ưu (CLRS Lemma 24.1):

> Mọi đường đi con của đường đi ngắn nhất cũng là đường đi ngắn nhất.

6.2 Relaxation (CLRS §24.3) – Kỹ thuật cốt lõi

Pseudocode 15: Relaxation

```
Relax(u, v, w):
    if d[v] > d[u] + w(u,v):
        d[v] = d[u] + w(u,v)
        pi[v] = u
```

$d[v]$ = **ước lượng trên** (upper bound) cho $\delta(s, v)$. Relaxation chỉ **giảm** $d[v]$.

6.3 Thuật toán Dijkstra (CLRS §24.3)

```
Dijkstra(G, w, s):
    for moi v in V:
        d[v] = ∞;  pi[v] = NIL
    d[s] = 0
    S = empty
    Q = MinPriorityQueue(V, d)
    while Q ≠ empty:
        u = ExtractMin(Q)
        S = S + {u}
        for moi v in Adj[u]:
            Relax(u, v, w)
```

Khởi tạo mọi đỉnh: khoảng cách $= \infty$, cha $=$ NIL

Đỉnh nguồn có $d[s] = 0$ – điểm xuất phát

***GREEDY:** Luôn chọn đỉnh gần nhất – bất biến cốt lõi*

***DP update:** Cập nhật $d[v]$ nếu tìm đường ngắn hơn qua u*

6.3.1 Chứng minh đúng đắn (CLRS Theorem 24.6):

Theorem: Khi u được thêm vào S (extracted from Q): $d[u] = \delta(s, u)$.

Chứng minh (phản chứng – CLRS style):

- Giả sử u là đỉnh **đầu tiên** thêm vào S mà $d[u] \neq \delta(s, u) \to d[u] > \delta(s, u)$
- Xét đường đi ngắn nhất $s \rightsquigarrow x \to y \rightsquigarrow u$, trong đó $x \in S, y \notin S$
- Vì x đã được xử lý: $d[x] = \delta(s, x) \to$ sau relax: $d[y] = \delta(s, y)$
- Vì u được chọn trước y (ExtractMin): $d[u] \leq d[y]$
- Mà $d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u]$ (vì $w \geq 0$)
- $\to d[u] = \delta(s, u)$ **mâu thuẫn!** (OK)

Điều kiện: $w(u, v) \geq 0$ cho mọi cạnh. Nếu có trọng số âm $\to$ dùng **Bellman-Ford**.

6.3.2 Minh họa trực quan Dijkstra (Visual Trace – đồ thị giống video YouTube):

Video hướng dẫn: Dijkstra's Algorithm – YouTube (click để xem)

Cho đồ thị **có hướng** G với $V = \{A, B, C, D, E\}$, nguồn $s = A$:

Cạnh	A→B	A→C	B→C	B→D	B→E	C→B	C→D	C→E	E→D
w	4	2	3	2	3	1	4	5	1

Table 9: Dijkstra – Danh sách cạnh đồ thị mẫu

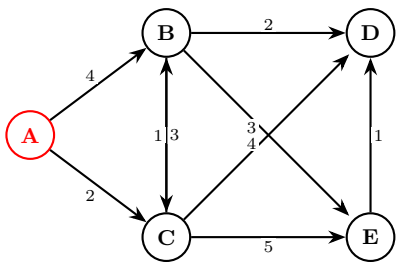


Figure 12: (a) Đồ thị gốc G – 9 cạnh có hướng

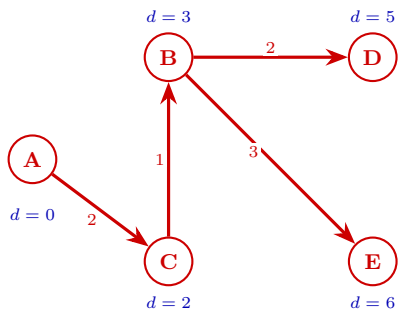


Figure 13: (b) Cây đường đi ngắn nhất từ A

Trace Dijkstra từ $s = A$:

Bước	$d[A]$	$d[B]$	$d[C]$	$d[D]$	$d[E]$	**ExtractMin**	**Relaxation**
Init	**[0]**	∞	∞	∞	∞	A	–
1	0	4	**[2]**	∞	∞	C	A→B: $d[B] = 4$, A→C: $d[C] = 2$
2	0	**[3]**	2	6	7	B	C→B: $\min(4, 2{+}1){=}3$, C→D: 6, C→E: 7
3	0	3	2	**[5]**	6	D	B→D: $\min(6, 3{+}2){=}5$, B→E: $\min(7, 3{+}3){=}6$
4	0	3	2	5	**[6]**	E	E→D: $6{+}1{=}7 > 5$, no change

Table 10: Dijkstra – Bảng trace khoảng cách từng bước

Kết quả: $d[A] = 0$, $d[B] = 3$, $d[C] = 2$, $d[D] = 5$, $d[E] = 6$.

Đường đi ngắn nhất:

- $A \to C$: chi phí $= 2$ (trực tiếp)
- $A \to C \to B$: chi phí $= 2 + 1 = 3$
- $A \to C \to B \to D$: chi phí $= 2 + 1 + 2 = 5$
- $A \to C \to B \to E$: chi phí $= 2 + 1 + 3 = 6$

6.3.3 Complexity (CLRS §24.3):

$O(V \lg V + E)$ (Fibonacci heap) hoặc $O((V + E) \lg V)$ (binary heap)

? Tại sao? Tại sao Dijkstra **không hoạt động** với cạnh có trọng số âm? Hãy tự xây dựng một **phản ví dụ** (3 đỉnh, 1 cạnh âm) mà Dijkstra cho kết quả sai.

▷ Tự Trace: Dijkstra – Trace bảng khoảng cách

Cho đồ thị có hướng 4 đỉnh $\{s, a, b, t\}$ với cạnh: $s{\to}a(1)$, $s{\to}b(4)$, $a{\to}b(2)$, $a{\to}t(6)$, $b{\to}t(3)$.

Bước	$d[s]$	$d[a]$	$d[b]$	$d[t]$	**ExtractMin**
Init	**[0]**	∞	∞	∞	s
1	0	_____	_____	_____	_____
2	0	_____	_____	_____	_____
3	0	_____	_____	_____	_____

Đường đi ngắn nhất $s \to t$: _____, chi phí = _____

---

7 Chương 7: Quy hoạch Động – Dynamic Programming (CLRS Ch.14)

Trực giác: DP = **"Nhớ để không tính lại"**. Nếu đệ quy ngây thơ giải Fibonacci tốn $O(2^n)$ vì tính trùng lắp, thì DP chỉ tốn $O(n)$ bằng cách lưu kết quả vào bảng. Khi nào dùng DP? Khi bài toán có **optimal substructure** và **overlapping subproblems** – hai tính chất quan trọng nhất. Floyd-Warshall và Matrix Chain Multiplication là hai ví dụ điển hình.

Sai lầm thường gặp: (1) DP chỉ đúng nếu có optimal substructure – không phải mọi bài "chia nhỏ" đều dùng DP được. (2) Trong Pascal/binomial coefficient DP, **phải duyệt j từ lớn đến nhỏ** (downto) để tránh dùng giá trị đã cập nhật trong cùng vòng lặp i. (3) Floyd-Warshall cần khởi tạo đường chéo $D[i][i] = 0$, không phải ∞.

7.1 Ý tưởng Quy hoạch Động (Dynamic Programming)

Quy hoạch Động (DP) giải bài toán tối ưu bằng cách **chia thành bài toán con chồng lấp** (overlapping subproblems), giải mỗi bài con **đúng 1 lần**, lưu kết quả vào bảng để tái sử dụng.

Khi nào dùng DP? Khi bài toán có 2 tính chất:

1. **Optimal substructure:** Nghiệm tối ưu bài toán lớn *chứa* nghiệm tối ưu bài toán con.

2. **Overlapping subproblems:** Các bài toán con được gọi lại nhiều lần (đệ quy ngây thơ sẽ tính lại → lãng phí).

Hai phương pháp triển khai:

- **Top-down (Memoization):** Viết đệ quy + cache. Dễ code nhưng overhead gọi hàm.
- **Bottom-up (Tabulation):** Điền bảng từ bài toán nhỏ → lớn. Hiệu quả hơn, không cần stack đệ quy.

So sánh D&C vs DP: D&C chia thành bài toán con *không chồng lấp* (independent), DP chia thành bài toán con *chồng lấp* (shared). Nếu có overlapping mà dùng D&C thẳng → exponential time (vd: Fibonacci naive $O(2^n)$).

7.2 Bốn bước thiết kế DP (CLRS §14.1)

1. **Characterize** cấu trúc nghiệm tối ưu (optimal substructure)
2. **Define** giá trị nghiệm tối ưu bằng đệ quy
3. **Compute** giá trị nghiệm tối ưu bottom-up (hoặc top-down + memo)
4. **Construct** nghiệm tối ưu từ thông tin đã tính

7.3 Hai tính chất then chốt (CLRS §14.3)

7.3.1 (1) Optimal Substructure:

Nghiệm tối ưu của bài toán **chứa** nghiệm tối ưu của bài toán con.

7.3.2 (2) Overlapping Subproblems:

Các bài toán con **trùng lặp** (gọi đi gọi lại) → lưu bảng thay vì tính lại.

7.3.3 So sánh: Tham lam – Chia để trị – Quy hoạch động

Tiêu chí	Tham lam (Greedy)	Chia để trị (D&C)	Quy hoạch động (DP)
Ý tưởng	Luôn chọn phương án **tốt nhất tại chỗ**, hy vọng dẫn đến tối ưu toàn cục	**Chia** thành bài con **độc lập**, giải đệ quy, rồi **gộp** kết quả	Chia thành bài con **chồng lấp**, giải **đúng 1 lần**, lưu bảng tái sử dụng
Bài toán con	Không chia rõ ràng – chỉ giảm kích thước sau mỗi chọn	**Tách biệt** (disjoint) – không giao nhau	**Chồng lấp** (overlapping) – cùng bài con xuất hiện nhiều lần
Tính chất cần	Greedy-choice property + Optimal substructure	Chia được thành bài con nhỏ cùng dạng	Optimal substructure + Overlapping subproblems
Thứ tự quyết định	**Trước** khi giải bài con (top-down, không quay lui)	Giải bài con **trước**, gộp sau	Giải **tất cả** bài con, sau đó chọn tối ưu
Cách giải	Vòng lặp đơn giản (sort + chọn)	Đệ quy trực tiếp	Tabulation (bottom-up) / Memoization (top-down)
Time phổ biến	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$ hoặc $O(n^3)$
Khi nào dùng?	Tối ưu + chứng minh được Greedy-choice	Bài con **độc lập** nhau	Tối ưu + bài con chồng lấp, *không* có Greedy-choice
Ví dụ	Kruskal, Prim, Dijkstra, Activity Selection	Merge Sort, Quick Sort, Binary Search	Fibonacci, Floyd-Warshall, MCM, 0/1 Knapsack

Table 12: So sánh ba phương pháp thiết kế thuật toán

7.4 Pascal – Tổ hợp $C(n, k)$

$C(n, k) = C(n-1, k) + C(n-1, k-1), \quad C(n, 0) = C(n, n) = 1$

7.4.1 Thuật toán 1: $O(nk)$ time, $O(nk)$ space

Pseudocode 17: Pascal Triangle – DP (O(nk) space)

```
Pascal1(n, k):
    C[0..n][0..k]
    for i = 0 to n:
        for j = 0 to min(i, k):
            if j = 0 or j = i: C[i][j] = 1
            else: C[i][j] = C[i-1][j] + C[i-1][j-1]
    return C[n][k]
```

Phân tích Complexity – Pascal Triangle DP ($O(nk)$ space)

Dòng code	Số lần chạy	Chi phí
`C[0..n][0..k]` khởi tạo	$(n+1)(k+1)$	cấp phát bảng 2D
`for i = 0 to n`	$n+1$	vòng ngoài
`  for j = 0 to min(i,k)`	$\sum_{i=0}^{n} \min(i,k) + 1$	vòng trong
`  C[i][j] = C[i-1][j] + C[i-1][j-1]`	$\leq nk$	$\Theta(1)$ mỗi ô

Time: 2 vòng lồng: i chạy $n+1$ lần, j chạy tối đa $\min(i, k) \to$ tổng $\leq n \cdot k$ ô $\to O(nk)$.
Space: Bảng 2D $C[0..n][0..k]$ có $(n+1)(k+1)$ ô $\to O(nk)$.

7.4.2 Thuật toán 2: $O(nk)$ time, $O(k)$ space – TỐI ƯU

Pseudocode 18: Pascal Triangle – DP tối ưu (O(k) space)

```
Pascal2(n, k):
    C[0..k], C[0] = 1
    for i = 1 to n:
        for j = min(i,k) downto 1:     // !!! PHAI duyet nguoc!
            C[j] = C[j] + C[j-1]
    return C[k]
```

Phân tích Complexity – Pascal Triangle DP tối ưu ($O(k)$ space)

Dòng code	Số lần chạy	Chi phí
`C[0..k], C[0] = 1`	$k+1$	cấp phát mảng 1D
`for i = 1 to n`	n	vòng ngoài
`  for j = min(i,k) downto 1`	$\leq nk$	**duyệt ngược** (quan trọng!)
`  C[j] = C[j] + C[j-1]`	$\leq nk$	$\Theta(1)$ mỗi lần

Time: Vẫn $O(nk)$ – cùng số phép tính như cách 1.
Space: Chỉ dùng 1 mảng $C[0..k] \to O(k)$! *Tối ưu từ $O(nk)$ xuống $O(k)$ nhờ duyệt **ngược** j.*
(Duyệt ngược đảm bảo $C[j-1]$ chưa bị ghi đè khi tính $C[j]$ – giống kỹ thuật 0/1 Knapsack.)

7.5 Floyd-Warshall – Đường đi ngắn nhất mọi cặp đỉnh (CLRS Ch.25)

Input: Ma trận $L[n \times n]$, $L[i][j] = w(v_i, v_j)$ hoặc ∞

Output: Ma trận $D[n \times n]$, $D[i][j] = \delta(i, j) =$ đường đi ngắn nhất $i \to j$

Optimal substructure: Đường đi ngắn nhất $i \to j$ qua tập đỉnh trung gian $\{1..k\}$ hoặc đi qua k, hoặc không.

Pseudocode 19: Floyd-Warshall

```
Floyd-Warshall(W):
    D = W
    for k = 1 to n:
        for i = 1 to n:
            for j = 1 to n:
                if D[i][j] > D[i][k] + D[k][j]:
                    D[i][j] = D[i][k] + D[k][j]
    return D
```

Phân tích Complexity – Floyd-Warshall

Dòng code	Số lần chạy	Chi phí
`D = W` (copy ma trận)	n^2	$\Theta(n^2)$
`for k = 1 to n`	n	vòng ngoài (đỉnh trung gian)
`  for i = 1 to n`	n^2	vòng giữa
`  for j = 1 to n`	n^3	vòng trong
`  D[i][j] = min(...)`	n^3	$\Theta(1)$ mỗi lần

Time: 3 vòng `for` lồng nhau, mỗi vòng n lần $\to n \times n \times n = \Theta(n^3)$.
Space: Ma trận $D[n \times n] \to \Theta(n^2)$. (Có thể cập nhật in-place trên 1 ma trận vì $D^{(k)}[i][k] = D^{(k-1)}[i][k]$.)

Ý nghĩa: $D^{(k)}[i][j]$ = đường đi ngắn nhất $i \to j$ chỉ dùng đỉnh trung gian $\{1, \ldots, k\}$

Complexity: $\Theta(n^3)$ time, $\Theta(n^2)$ space

? Tại sao? Trong Floyd-Warshall, tại sao vòng lặp k **phải ở ngoài cùng**? Nếu đặt i hoặc j ở ngoài thì sao? Gợi ý: nghĩ về "qua tập đỉnh trung gian $\{1..k\}$" – thứ tự xây bảng DP quan trọng!

7.5.1 Ví dụ minh họa: 4 đỉnh (giải đầy đủ)

Đồ thị ban đầu (Input)

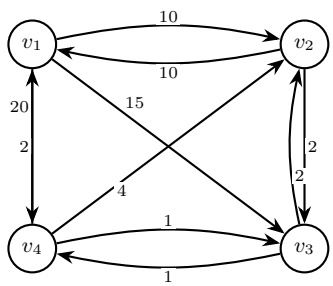


Figure 14: (a) Đồ thị gốc – 10 cạnh có hướng

Shortest Paths từ v_1 (Output)

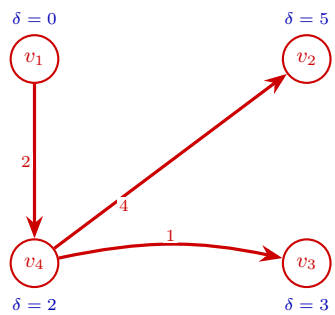


Figure 15: (b) Cây đường đi ngắn nhất từ v_1
$v_1 \to v_4 \to v_2 = 5, \quad v_1 \to v_4 \to v_3 = 3$

Ma trận trọng số L (input):

$$D^{(0)} = L = \begin{pmatrix} 0 & 10 & 15 & 2 \\ 10 & 0 & 2 & \infty \\ \infty & 2 & 0 & 1 \\ 20 & 4 & 1 & 0 \end{pmatrix}$$

$k = 1$ **(xét đỉnh trung gian v_1):** Kiểm tra $D[i][j]$ so với $D[i][1] + D[1][j]$.

$D[2][4] = \min(\infty, 10 + 2) = 12, \quad D[4][2] = \min(4, 20 + 10) = 4$ (không đổi)

$$D^{(1)} = \begin{pmatrix} 0 & 10 & 15 & 2 \\ 10 & 0 & 2 & 12 \\ \infty & 2 & 0 & 1 \\ 20 & 4 & 1 & 0 \end{pmatrix}$$

$k = 2$ **(qua v_1, v_2):**

$D[1][3] = \min(15, 10 + 2) = 12, \quad D[3][1] = \min(\infty, 2 + 10) = 12$

$D[3][4] = \min(1, 2 + 12) = 1$ (không đổi), $\quad D[4][3] = \min(1, 4 + 2) = 1$ (không đổi)

$$D^{(2)} = \begin{pmatrix} 0 & 10 & 12 & 2 \\ 10 & 0 & 2 & 12 \\ 12 & 2 & 0 & 1 \\ 20 & 4 & 1 & 0 \end{pmatrix}$$

$k = 3$ **(qua v_1, v_2, v_3):**

$D[1][4] = \min(2, 12 + 1) = 2$ (không đổi), $\quad D[2][4] = \min(12, 2 + 1) = 3$

$D[4][1] = \min(20, 1 + 12) = 13, \quad D[4][2] = \min(4, 1 + 2) = 3$

$$D^{(3)} = \begin{pmatrix} 0 & 10 & 12 & 2 \\ 10 & 0 & 2 & 3 \\ 12 & 2 & 0 & 1 \\ 13 & 3 & 1 & 0 \end{pmatrix}$$

$k = 4$ **(qua tất cả đỉnh):**

$D[1][2] = \min(10, 2 + 3) = 5, \quad D[1][3] = \min(12, 2 + 1) = 3$

$D[2][1] = \min(10, 3 + 13) = 10$ (không đổi), $\quad D[3][1] = \min(12, 1 + 13) = 12$ (không đổi)

$$\boxed{D^{(4)} = \begin{pmatrix} 0 & 5 & 3 & 2 \\ 10 & 0 & 2 & 3 \\ 12 & 2 & 0 & 1 \\ 13 & 3 & 1 & 0 \end{pmatrix}}$$

Đọc kết quả: $\delta(v_1, v_2) = 5$ (đi $v_1 \to v_4 \to v_2$), $\delta(v_1, v_3) = 3$ (đi $v_1 \to v_4 \to v_3$).

7.5.2 Ví dụ 2: 4 đỉnh với cạnh âm

Lưu ý: Đồ thị này có **cạnh âm** (-2 và -1) – Dijkstra **không hoạt động** nhưng Floyd-Warshall vẫn đúng!

Đồ thị ban đầu (Input)

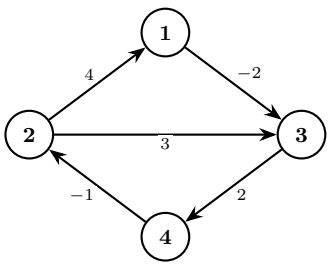


Figure 16: (a) Đồ thị có cạnh âm – 5 cạnh

Shortest Paths từ đỉnh 1

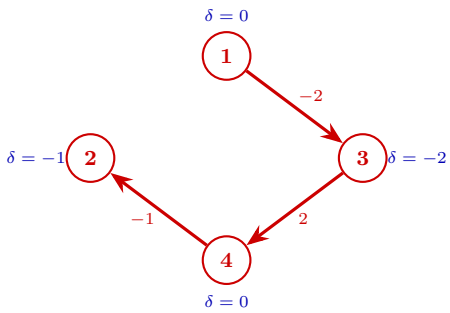


Figure 17: (b) 1→3→4→2 = −1

Ma trận trọng số W (input), ∞ = không có cạnh:

$$D^{(0)} = W = \begin{pmatrix} 0 & \infty & -2 & \infty \\ 4 & 0 & 3 & \infty \\ \infty & \infty & 0 & 2 \\ \infty & -1 & \infty & 0 \end{pmatrix}$$

$k = 1$ **(qua đỉnh 1):**

$D[2][3] = \min(3, 4 + (-2)) = \min(3, 2) = 2$

$$D^{(1)} = \begin{pmatrix} 0 & \infty & -2 & \infty \\ 4 & 0 & \mathbf{2} & \infty \\ \infty & \infty & 0 & 2 \\ \infty & -1 & \infty & 0 \end{pmatrix}$$

$k = 2$ **(qua đỉnh 1, 2):**

$D[4][1] = \min(\infty, -1 + 4) = 3, \quad D[4][3] = \min(\infty, -1 + 2) = 1$

$$D^{(2)} = \begin{pmatrix} 0 & \infty & -2 & \infty \\ 4 & 0 & 2 & \infty \\ \infty & \infty & 0 & 2 \\ \mathbf{3} & -1 & \mathbf{1} & 0 \end{pmatrix}$$

$k = 3$ **(qua đỉnh 1, 2, 3):**

$D[1][4] = \min(\infty, -2 + 2) = 0, \quad D[2][4] = \min(\infty, 2 + 2) = 4$

$$D^{(3)} = \begin{pmatrix} 0 & \infty & -2 & \mathbf{0} \\ 4 & 0 & 2 & \mathbf{4} \\ \infty & \infty & 0 & 2 \\ 3 & -1 & 1 & 0 \end{pmatrix}$$

$k = 4$ **(qua tất cả đỉnh):**

$D[1][2] = \min(\infty, 0 + (-1)) = -1, \quad D[2][1] = \min(4, 4 + 3) = 4$ (không đổi)

$D[3][1] = \min(\infty, 2 + 3) = 5, \quad D[3][2] = \min(\infty, 2 + (-1)) = 1$

$$\boxed{D^{(4)} = \begin{pmatrix} 0 & -1 & -2 & 0 \\ 4 & 0 & 2 & 4 \\ 5 & 1 & 0 & 2 \\ 3 & -1 & 1 & 0 \end{pmatrix}}$$

Đọc kết quả từ đỉnh 1: $\delta(1, 2) = -1$ (đi $1 \to 3 \to 4 \to 2$), $\delta(1, 3) = -2$ (đi $1 \to 3$), $\delta(1, 4) = 0$ (đi $1 \to 3 \to 4$).

Nhận xét: Đường đi ngắn nhất có thể **âm** nhờ cạnh âm. Floyd-Warshall xử lý đúng vì xét *tất cả* đỉnh trung gian, không Greedy.

---

8 Chương 8: Nhân Chuỗi Ma trận – Matrix Chain Multiplication (CLRS §14.2)

Trực giác: Nhân ma trận có tính kết hợp: $(AB)C = A(BC)$. Nhưng **chi phí tính toán** lại phụ thuộc cách đặt ngoặc! Đây là bài toán tối ưu hóa thuần túy – không thể Greedy (không có greedy-choice đơn giản) – phải DP. Ta điền bảng $m[i][j]$ theo thứ tự tăng dần độ dài chain, từng bước xây dựng lên nghiệm tối ưu.

Sai lầm thường gặp: (1) Chỉ số p có $n+1$ phần tử cho n ma trận: $p = \langle p_0, p_1, \ldots, p_n \rangle$. (2) Điền bảng **theo chain length** ($l = 2, 3, \ldots, n$) – không phải theo thứ tự i, j. (3) $m[i][j]$ lưu **số phép nhân scalar**, không phải số phép nhân ma trận.

8.1 8.1 Bài toán

Cho dãy ma trận $\langle A_1, A_2, \ldots, A_n \rangle$, A_i có kích thước $p_{i-1} \times p_i$. Tìm cách đặt ngoặc để **tổng số phép nhân đơn vị** nhỏ nhất.

Ví dụ: $A(3 \times 100)$, $B(100 \times 50)$, $C(50 \times 20)$:

- $(AB)C = 15{,}000 + 3{,}000 = 18{,}000$ (tốt)
- $A(BC) = 100{,}000 + 6{,}000 = 106{,}000$ (xấu)

8.2 Phân tích DP (CLRS §14.2)

Optimal substructure: Nếu đặt ngoặc tối ưu chia tại k: $(A_i \ldots A_k)(A_{k+1} \ldots A_j)$, thì mỗi phần cũng phải được đặt ngoặc tối ưu.

Công thức:

$$m[i][j] = \begin{cases} 0 & \text{nếu } i = j \\ \min\limits_{i \le k < j} \{m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j\} & \text{nếu } i < j \end{cases}$$

Số Catalan: Số cách đặt ngoặc $= C(n) = \Omega(4^n / n^{3/2}) \to$ brute force **không khả thi**.

8.2.1 Thuật toán (CLRS):

Pseudocode 20: Matrix Chain Multiplication

```
MatrixChainOrder(p):
    n = len(p) - 1
    m[1..n][1..n],  s[1..n][1..n]
    for i = 1 to n: m[i][i] = 0
    for l = 2 to n:                    // chain length
        for i = 1 to n-l+1:
            j = i + l - 1
            m[i][j] = INF
            for k = i to j-1:
                q = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]
                if q < m[i][j]:
                    m[i][j] = q
                    s[i][j] = k        // luu split point
    return m, s

PrintOptimalParens(s, i, j):
    if i = j: print "A_i"
    else:
        print "("
        PrintOptimalParens(s, i, s[i][j])
        PrintOptimalParens(s, s[i][j]+1, j)
        print ")"
```

Phân tích Complexity – Matrix Chain Multiplication

Dòng code	Số lần chạy	Chi phí
`m[1..n][1..n], s[1..n][1..n]`	$2n^2$	cấp phát 2 bảng
`for i = 1 to n: m[i][i]=0`	n	khởi tạo đường chéo
`for l = 2 to n`	$n-1$	chain length (vòng ngoài)
`  for i = 1 to n-l+1`	$\sum_{l=2}^{n}(n-l+1)$	vòng giữa
`  for k = i to j-1`	$\sum_{l=2}^{n}\sum_{i}(l-1)$	vòng trong (thử split)
`  q = m[i][k]+m[k+1][j]+p...`	mỗi lần $\Theta(1)$	1 phép cộng + nhân

Tính tổng vòng lặp:

$$\sum_{l=2}^{n}\sum_{i=1}^{n-l+1}(l-1) = \sum_{l=2}^{n}(n-l+1)(l-1) = \Theta(n^3)$$

(Đây là tổ hợp $\binom{n}{3}$ – số cách chọn 3 chỉ số từ n.)
Time: 3 vòng lồng nhau $\to \Theta(n^3)$.
Space: Bảng $m[n \times n] + s[n \times n] \to \Theta(n^2)$.

8.2.2 Ví dụ minh họa: 3 ma trận

Cho $A_1(10 \times 30)$, $A_2(30 \times 5)$, $A_3(5 \times 60)$. Vậy $p = \langle 10, 30, 5, 60 \rangle$.

Chain dài 2 ($l = 2$):

$m[1][2] = p_0 \cdot p_1 \cdot p_2 = 10 \times 30 \times 5 = 1500, \quad s[1][2] = 1$

$m[2][3] = p_1 \cdot p_2 \cdot p_3 = 30 \times 5 \times 60 = 9000, \quad s[2][3] = 2$

Chain dài 3 ($l = 3$):

$$m[1][3] = \min \begin{cases} k{=}1 : m[1][1] + m[2][3] + p_0p_1p_3 = 0 + 9000 + 10 \times 30 \times 60 = 27000 \\ k{=}2 : m[1][2] + m[3][3] + p_0p_2p_3 = 1500 + 0 + 10 \times 5 \times 60 = 4500 \end{cases}$$

$m[1][3] = \boxed{4500}, \quad s[1][3] = 2$

Bảng m:

m	1	2	3
1	0	1500	**4500**
2		0	9000
3			0

Thứ tự tối ưu: $s[1][3] = 2$, nên chia $(A_1A_2)(A_3) = \boxed{(A_1A_2)A_3}$.

Chi phí: nhân A_1A_2 tốn 1500, kết quả 10×5; rồi nhân với A_3 tốn $10 \times 5 \times 60 = 3000$. Tổng: 4500.

So sánh: $A_1(A_2A_3)$ tốn $9000 + 18000 = 27000$ – **gấp 6 lần!**

---

9 Chương 9: Lý thuyết Độ phức tạp – P, NP, NP-Complete (CLRS Ch.34)

Trực giác: P vs NP là câu hỏi mở lớn nhất của khoa học máy tính (và là 1 trong 7 bài toán Millennium Prize). Trực giác: P = "giải được nhanh", NP = "kiểm tra được nhanh". Chứng minh một bài toán NP-Complete có nghĩa là **không ai** tìm được thuật toán polynomial cho nó – một kết quả tiêu cực nhưng rất hữu ích, vì ta biết nên chuyển sang tìm approximation thay vì tìm nghiệm chính xác.

Sai lầm thường gặp: (1) NP không có nghĩa là "Non-Polynomial" – NP = Nondeterministic Polynomial (kiểm tra được trong poly time). (2) $P \subseteq NP$ nhưng ta chưa biết $P = NP$ hay $P \neq NP$. (3) Để chứng minh L là NP-Complete, phải chứng minh cả 2 điều: $L \in NP$ VÀ có reduction từ một bài NPC đã biết vào L (không phải từ L ra).

9.1 9.1 Lớp P

$$\mathbf{P} = \{\text{bài toán quyết định giải được trong } O(n^k) \text{ cho hằng số } k\}$$

Ví dụ: Sorting $O(n \lg n)$, MST $O(E \lg V)$, Shortest path $O(V \lg V + E)$

9.2 9.2 Lớp NP

$$\mathbf{NP} = \{\text{bài toán quyết định mà } \textbf{lời giải kiểm tra được} \text{ trong polynomial time}\}$$

- Cho certificate (lời giải), verifier kiểm tra trong $O(n^k)$
- $P \subseteq NP$ (hiển nhiên: giải được $\rightarrow$ kiểm tra được)
- $P = NP$? $\rightarrow$ Câu hỏi mở lớn nhất của CS!

9.3 Polynomial Reduction (CLRS §34.3)

$A \leq_p B$ (A quy về B): Có hàm f tính trong polynomial time sao cho:

$$x \in A \iff f(x) \in B$$

Ý nghĩa: B **ít nhất khó bằng** A. Nếu giải B được $\rightarrow$ giải A được.

9.4 NP-Complete (CLRS §34.4)

Bài toán L là **NP-Complete** nếu:

1. $L \in NP$
2. $\forall L' \in NP$: $L' \leq_p L$ (mọi bài toán NP đều quy về L)

Ví dụ: Hamilton Cycle – đi qua tất cả đỉnh đúng 1 lần, quay về.

- $\in NP$: cho chu trình, kiểm tra $O(n)$
- Chưa có thuật toán polynomial để **tìm** $\rightarrow$ NP-Complete
- Nếu ai giải được Hamilton trong P $\rightarrow$ $P = NP$!

Cách chứng minh NPC (CLRS §34.4):

1. Chứng minh $L \in NP$
2. Chọn bài NPC đã biết L' (SAT, 3-SAT, CLIQUE, VERTEX-COVER, HAMILTONIAN-CYCLE...)
3. Chứng minh $L' \leq_p L$ (polynomial reduction)
4. $\rightarrow$ L là NP-Complete (OK)

⇒ Giải thích bằng lời bạn: Giải thích P vs NP cho bà ngoại bạn. Gợi ý: "P = bài toán mà máy tính giải NHANH. NP = bài toán mà máy tính KIỂM TRA lời giải nhanh. Câu hỏi triệu đô: có phải mọi bài kiểm tra nhanh đều giải nhanh?" – Nếu bạn giải thích được rõ hơn thế, bạn đã hiểu.

✓ Tự kiểm tra: Để chứng minh bài toán L là NP-Complete, ta cần reduce TỪ bài NPC đã biết L' VÀO L (tức $L' \leq_p L$). Tại sao chiều reduction quan trọng? Nếu reduce ngược ($L \leq_p L'$) thì ta chứng minh được điều gì?

? Tại sao? Tại sao một số bài toán **không thể giải được trong thời gian đa thức** (polynomial time)? Nếu P $\neq$ NP, điều đó có ý nghĩa gì cho thực tiễn? Gợi ý: nghĩ về số lượng nghiệm cần kiểm tra và tại sao "kiểm tra nhanh" không đồng nghĩa với "tìm nhanh".

10 Chương 10: Bài tập Mẫu có Lời giải – Practice Problems with Solutions

10.1 Bài 1: Độ phức tạp – Master Theorem

Đề bài: Giải các công thức đệ quy sau:

(a) $T(n) = 4T(n/2) + n$
(b) $T(n) = 2T(n/2) + n \lg n$
(c) $T(n) = 3T(n/4) + n \lg n$

Lời giải:

(a) $T(n) = 4T(n/2) + n$

$a = 4, \quad b = 2, \quad \log_b a = \log_2 4 = 2$

$f(n) = n = O(n^{2-\varepsilon})$ với $\varepsilon = 1$. Vì $f(n)$ tăng chậm hơn n^2:

$\rightarrow$ **Case 1:** $T(n) = \Theta(n^{\log_2 4}) = \boxed{\Theta(n^2)}$

(b) $T(n) = 2T(n/2) + n \lg n$

$a = 2, \quad b = 2, \quad \log_b a = 1$

$f(n) = n \lg n = \Theta(n^1 \cdot \lg^1 n)$ với $k = 1$.

$\rightarrow$ **Case 2:** $T(n) = \Theta(n^1 \cdot \lg^{1+1} n) = \boxed{\Theta(n \lg^2 n)}$

(c) $T(n) = 3T(n/4) + n \lg n$

$a = 3, \quad b = 4, \quad \log_4 3 \approx 0.793$

$f(n) = n \lg n = \Omega(n^{0.793+\varepsilon})$ với $\varepsilon \approx 0.2$.

Kiểm tra regularity: $3 \cdot (n/4) \lg(n/4) = (3/4)n \lg(n/4) \leq (3/4)n \lg n = c \cdot f(n)$ với $c = 3/4 < 1$. (OK)

$\rightarrow$ **Case 3:** $T(n) = \Theta(f(n)) = \boxed{\Theta(n \lg n)}$

10.2 Bài 2: QuickSort – Trace Partition

Đề bài: Cho mảng $A = [7, 2, 1, 6, 8, 5, 3, 4]$. Thực hiện Partition với pivot $= A[8] = 4$. Cho biết trạng thái mảng sau mỗi bước.

Lời giải:

Pivot $= A[8] = 4$, $i = 0$ (trước p).

j	A[j]	So sánh	Mảng sau bước
1	7	$7 > 4 \rightarrow$ skip	$[7, 2, 1, 6, 8, 5, 3, \mathbf{4}]$, $i = 0$
2	2	$2 \leq 4 \rightarrow i$=1, swap $A[1] \leftrightarrow A[2]$	$[\mathbf{2}, 7, 1, 6, 8, 5, 3, 4]$, $i = 1$
3	1	$1 \leq 4 \rightarrow i$=2, swap $A[2] \leftrightarrow A[3]$	$[2, \mathbf{1}, 7, 6, 8, 5, 3, 4]$, $i = 2$
4	6	$6 > 4 \rightarrow$ skip	không đổi, $i = 2$
5	8	$8 > 4 \rightarrow$ skip	không đổi, $i = 2$
6	5	$5 > 4 \rightarrow$ skip	không đổi, $i = 2$
7	3	$3 \leq 4 \rightarrow i$=3, swap $A[3] \leftrightarrow A[7]$	$[2, 1, \mathbf{3}, 6, 8, 5, 7, 4]$, $i = 3$

Swap cuối: $A[i+1] \leftrightarrow A[r] = A[4] \leftrightarrow A[8]$:

$$A = [2, 1, 3, \boxed{4}, 8, 5, 7, 6]$$

Kết quả: pivot 4 ở vị trí $q = 4$.

- Bên trái ($A[1..3] = [2, 1, 3]$): tất cả ≤ 4 (OK)

- Bên phải ($A[5..8] = [8, 5, 7, 6]$): tất cả > 4 (OK)

Complexity: Partition duyệt $n - 1$ phần tử $\rightarrow \Theta(n)$.

10.3 Bài 4: Dijkstra – Đường đi ngắn nhất

Đề bài: Cho đồ thị có hướng G với 5 đỉnh $\{s, a, b, c, t\}$ và các cạnh:

Cạnh	$s \rightarrow a$	$s \rightarrow b$	$a \rightarrow b$	$a \rightarrow c$	$b \rightarrow c$	$b \rightarrow t$	$c \rightarrow t$
w	2	5	1	7	3	8	1

Tìm đường đi ngắn nhất từ s đến tất cả các đỉnh.

Lời giải:

Khởi tạo: $d[s] = 0$, $d[a] = d[b] = d[c] = d[t] = \infty$.

Bước	$d[s]$	$d[a]$	$d[b]$	$d[c]$	$d[t]$	**ExtractMin**	**Relaxation**
Init	**[0]**	∞	∞	∞	∞	s	
1	0	**[2]**	5	∞	∞	a	$d[a]$=0+2=2, $d[b]$=0+5=5
2	0	2	**[3]**	9	∞	b	$d[b]=\min(5, 2+1)=3$, $d[c]$=2+7=9
3	0	2	3	**[6]**	11	c	$d[c]=\min(9, 3+3)=6$, $d[t]$=3+8=11
4	0	2	3	6	**[7]**	t	$d[t]=\min(11, 6+1)=7$

Kết quả:

Đỉnh	$\delta(s, v)$	**Đường đi**
s	0	s
a	2	$s \rightarrow a$
b	3	$s \rightarrow a \rightarrow b$
c	6	$s \rightarrow a \rightarrow b \rightarrow c$
t	7	$s \rightarrow a \rightarrow b \rightarrow c \rightarrow t$

Nhận xét: Đường $s \rightarrow t$ trực tiếp qua b tốn $5 + 8 = 13$, nhưng đi $s \rightarrow a \rightarrow b \rightarrow c \rightarrow t$ chỉ tốn $\boxed{7}$.

10.4 Bài 5: Quy hoạch động – Floyd-Warshall

Đề bài: Cho đồ thị 4 đỉnh với ma trận trọng số:

$$L = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix}$$

Tìm ma trận đường đi ngắn nhất D bằng Floyd-Warshall.

Lời giải:

Khởi tạo $D^{(0)} = L$. Công thức: $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$

$k = 1$ **(qua đỉnh 1):**

$D[2][4] = \min(\infty, D[2][1] + D[1][4]) = \min(\infty, 8 + 7) = 15$

$D[3][2] = \min(\infty, D[3][1] + D[1][2]) = \min(\infty, 5 + 3) = 8$

$D[3][4] = \min(1, D[3][1] + D[1][4]) = \min(1, 5 + 7) = 1$ (không đổi)

$$D^{(1)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{pmatrix}$$

$k = 2$ **(qua đỉnh 1, 2):**

$D[1][3] = \min(\infty, D[1][2] + D[2][3]) = \min(\infty, 3 + 2) = 5$

$D[3][4] = \min(1, D[3][2] + D[2][4]) = \min(1, 8 + 15) = 1$ (không đổi)

$D[4][3] = \min(\infty, D[4][2] + D[2][3]) = \min(\infty, 5 + 2) = 7$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

$k = 3$ **(qua đỉnh 1, 2, 3):**

$D[1][4] = \min(7, D[1][3] + D[3][4]) = \min(7, 5 + 1) = 6$

$D[2][4] = \min(15, D[2][3] + D[3][4]) = \min(15, 2 + 1) = 3$

$D[4][4] = 0$ (không đổi)

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ 8 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

$k = 4$ **(qua tất cả đỉnh):**

$D[1][2] = \min(3, D[1][4] + D[4][2]) = \min(3, 6 + 5) = 3$ (không đổi)

$D[2][1] = \min(8, D[2][4] + D[4][1]) = \min(8, 3 + 2) = 5$

$D[3][1] = \min(5, D[3][4] + D[4][1]) = \min(5, 1 + 2) = 3$

$D[3][2] = \min(8, D[3][4] + D[4][2]) = \min(8, 1 + 5) = 6$

$$\boxed{D^{(4)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}}$$

10.5 Bài 6: Quy hoạch động – Nhân chuỗi ma trận

Đề bài: Cho 4 ma trận $A_1(5 \times 4)$, $A_2(4 \times 6)$, $A_3(6 \times 2)$, $A_4(2 \times 7)$.

Tìm thứ tự nhân tối ưu. Cho $p = \langle 5, 4, 6, 2, 7 \rangle$.

Lời giải:

$m[i][i] = 0$ với mọi i. Công thức: $m[i][j] = \min_{i \le k < j}\{m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j\}$

Chain dài 2 ($l = 2$)**:**

$m[1][2] = m[1][1] + m[2][2] + p_0 p_1 p_2 = 0 + 0 + 5 \times 4 \times 6 = \boxed{120}, \quad s[1][2] = 1$

$m[2][3] = 0 + 0 + 4 \times 6 \times 2 = \boxed{48}, \quad s[2][3] = 2$

$m[3][4] = 0 + 0 + 6 \times 2 \times 7 = \boxed{84}, \quad s[3][4] = 3$

Chain dài 3 ($l = 3$)**:**

$$m[1][3] = \min \begin{cases} k{=}1 : m[1][1] + m[2][3] + 5 \times 4 \times 2 = 0 + 48 + 40 = 88 \\ k{=}2 : m[1][2] + m[3][3] + 5 \times 6 \times 2 = 120 + 0 + 60 = 180 \end{cases} = \boxed{88}, \quad s[1][3] = 1$$

$$m[2][4] = \min \begin{cases} k{=}2 : m[2][2] + m[3][4] + 4 \times 6 \times 7 = 0 + 84 + 168 = 252 \\ k{=}3 : m[2][3] + m[4][4] + 4 \times 2 \times 7 = 48 + 0 + 56 = 104 \end{cases} = \boxed{104}, \quad s[2][4] = 3$$

Chain dài 4 ($l = 4$):

$$m[1][4] = \min \begin{cases} k{=}1 : m[1][1] + m[2][4] + 5 \times 4 \times 7 = 0 + 104 + 140 = 244 \\ k{=}2 : m[1][2] + m[3][4] + 5 \times 6 \times 7 = 120 + 84 + 210 = 414 \\ k{=}3 : m[1][3] + m[4][4] + 5 \times 2 \times 7 = 88 + 0 + 70 = 158 \end{cases}$$

$m[1][4] = \boxed{158}, \quad s[1][4] = 3$

Bảng m:

m	1	2	3	4
1	0	120	88	**158**
2		0	48	104
3			0	84
4				0

Thứ tự nhân tối ưu: Từ $s[1][4] = 3$: $(A_1A_2A_3)(A_4)$. Từ $s[1][3] = 1$: $(A_1)(A_2A_3)$.

$\boxed{(A_1(A_2A_3))A_4}$ Chi phí: 158 phép nhân

---

11 Chương 11: Cheat Sheet – Tổng hợp

11.1 Bảng Complexity

Thuật toán	Time	Space	Ghi chú
Insertion Sort	$\Theta(n^2)$, best $\Theta(n)$	$O(1)$	Stable, in-place
Merge Sort	$\Theta(n \lg n)$	$\Theta(n)$	Stable, optimal
Quick Sort	$\Theta(n \lg n)$ avg, $\Theta(n^2)$ worst	$O(\lg n)$	Nhanh nhất thực tế
Kruskal	$O(E \lg V)$	$O(V)$	MST, Union-Find
Prim	$O(E \lg V)$	$O(V)$	MST, Priority Queue
Dijkstra	$O((V + E) \lg V)$	$O(V)$	SSSP, $w \geq 0$
Floyd-Warshall	$\Theta(V^3)$	$\Theta(V^2)$	All-pairs SP
Matrix Chain	$\Theta(n^3)$	$\Theta(n^2)$	DP
Fib DP	$\Theta(n)$	$O(1)$	Bottom-up
Fib Matrix	$\Theta(\lg n)$	$O(1)$	Nhanh nhất

Table 13: Cheat Sheet – Bảng tổng hợp Complexity tất cả thuật toán

11.2 Công thức cốt lõi

$$\text{Relaxation: } d[v] = \min(d[v],\ d[u] + w(u, v))$$

$$\text{Floyd: } D[i][j] = \min(D[i][j],\ D[i][k] + D[k][j])$$

$$\text{MCM: } m[i][j] = \min_k \{m[i][k] + m[k + 1][j] + p_{i-1} \cdot p_k \cdot p_j\}$$

$$\text{Pascal: } C(n, k) = C(n - 1, k) + C(n - 1, k - 1)$$

$$\text{Fibonacci: } F(n) = F(n - 1) + F(n - 2)$$

$$\text{Cây: } |E| = |V| - 1$$

$$\text{Handshaking: } \sum \deg(v) = 2|E|$$

$$\text{Cận dưới sắp xếp so sánh: } \Omega(n \lg n)$$

11.3 Template giải bài

11.3.1 Template Master Theorem

Pseudocode 21: Template Master Theorem

```
1. Viet T(n) = aT(n/b) + f(n)
2. Tinh log_b(a)
3. So sanh f(n) voi n^(log_b a):
   polynomially smaller -> Case 1 -> Theta(n^(log_b a))
   same (x lg^k n)      -> Case 2 -> Theta(n^(log_b a) * lg^(k+1) n)
   polynomially larger   -> Case 3 -> Theta(f(n))  [kiem tra regularity]
```

11.3.2 Template Dijkstra

Pseudocode 22: Template Dijkstra

```
1. d[s]=0, d[v]=INF voi moi v!=s, Q = all vertices
2. Loop: u = ExtractMin(Q)
3. For moi v ke u: Relax(u,v,w)
4. Dien bang distance theo moi buoc
```

11.3.3 Template MST

Pseudocode 23: Template MST

```
Kruskal: Sap canh tang dan, them neu khong chu trinh, dung (n-1) canh
Prim:    Chon s, moi buoc them canh nhe nhat ra ngoai cay, dung (n-1) canh
```

11.3.4 Template chứng minh NP-Complete

Pseudocode 24: Template chứng minh NP-Complete

```
1. Chung minh in NP (cho certificate, verify trong poly time)
2. Chon bai NPC da biet L'
3. Chung minh L' <=_p L (xay ham reduction f, chung minh x in L' <=> f(x) in L)
```

12 Chương 12: Lời giải – Tại sao? & Tự Trace

Phần này cung cấp lời giải chi tiết cho tất cả các hộp **"? Tại sao?"** và **"▷ Tự Trace"** trong tài liệu, được đánh chỉ mục theo chương.

12.1 Chương 1: Nền tảng

12.1.1 Tại sao? 1.1 – Tại sao không dùng đo thời gian thực tế thay cho Big-O?

Câu hỏi: Tại sao ta không dùng đo thời gian chạy thực tế (empirical timing) thay cho Big-O?

Trả lời:

1. **Phụ thuộc phần cứng:** Cùng thuật toán, máy i9 chạy nhanh hơn máy Pentium gấp 100 lần – thời gian thực tế không phản ánh bản chất thuật toán.
2. **Phụ thuộc input:** Thời gian chạy trên mảng 100 phần tử không nói gì về mảng 10^6 phần tử. Big-O cho biết *tốc độ tăng* khi input lớn dần.
3. **Tính phổ quát:** Big-O là ngôn ngữ chung, độc lập với ngôn ngữ lập trình, compiler, và hệ điều hành. $O(n \lg n)$ luôn tốt hơn $O(n^2)$ khi n đủ lớn, bất kể máy nào.
4. **Worst-case guarantee:** Đo thực tế chỉ cho kết quả trên 1 input cụ thể. Big-O đảm bảo cho **mọi** input.

12.1.2 Tự Trace 1.1 – Loop Invariant: Điền 3 bước

Câu hỏi: Chứng minh Insertion Sort đúng bằng Loop Invariant.

Trả lời:

1. **Bất biến:** "Tại đầu mỗi lần lặp for với j, *mảng con* $A[1..j-1]$ *chứa các phần tử ban đầu của* $A[1..j-1]$ *nhưng đã được sắp xếp tăng dần.*"
2. **Initialization ($j = 2$):** Mảng con $A[1..1]$ chỉ có 1 phần tử → đã sắp xếp. Bất biến đúng.
3. **Maintenance:** Giả sử $A[1..j-1]$ đã sắp. Khi xét $key = A[j]$, ta dịch các phần tử $> key$ sang phải và chèn key vào đúng vị trí. Sau bước này, $A[1..j]$ đã sắp. Bất biến đúng trước lần lặp $j + 1$.
4. **Termination:** Vòng lặp dừng khi $j = n + 1$. Bất biến cho ta: $A[1..n]$ đã sắp tăng dần. □

12.2 Chương 2: Thuật toán Sắp xếp

12.2.1 Tại sao? 2.1 – Merge Sort cần extra space mà Quick Sort thì không?

Câu hỏi: Tại sao Merge Sort cần $\Theta(n)$ extra space mà Quick Sort thì không?

Trả lời:

- **Merge Sort:** Bước `Merge` gộp 2 mảng con *đã sắp* thành 1 mảng sắp. Để gộp, ta phải tạo mảng tạm chứa kết quả, vì không thể merge tại chỗ mà vẫn giữ $O(n)$. Merge in-place tồn tại nhưng phức tạp $O(n \lg n)$ và hằng số rất lớn – thực tế không ai dùng.
- **Quick Sort:** Partition chỉ *hoán đổi* phần tử trong chính mảng A (swap $A[i] \leftrightarrow A[j]$), không cần mảng phụ. Chỉ dùng $O(\lg n)$ stack cho đệ quy.

12.2.2 Tự Trace 2.1 – Insertion Sort: Trace mảng $[5, 3, 8, 1, 4]$

Câu hỏi: Trace Insertion Sort trên $A = [5, 3, 8, 1, 4]$.

Trả lời:

j	key	Mảng A sau bước	
–	–	$[5, 3, 8, 1, 4]$	
2	3	$[\mathbf{3}, 5, 8, 1, 4]$	($3 < 5$, dịch 5 sang phải, chèn 3)
3	8	$[3, 5, \mathbf{8}, 1, 4]$	($8 > 5$, giữ nguyên)
4	1	$[\mathbf{1}, 3, 5, 8, 4]$	($1 <$ tất cả, dịch 3 phần tử, chèn 1 đầu)
5	4	$[1, 3, \mathbf{4}, 5, 8]$	($4 > 3$, $4 < 5$, chèn giữa)

12.2.3 Tại sao? 2.2 – Counting Sort không vi phạm cận dưới $\Omega(n \lg n)$?

Câu hỏi: Tại sao Counting Sort chạy $O(n + k)$ mà không vi phạm cận dưới $\Omega(n \lg n)$?

Trả lời: Cận dưới $\Omega(n \lg n)$ chỉ áp dụng cho thuật toán **dựa trên so sánh** (comparison-based), tức thuật toán chỉ dùng phép so sánh $<, >, =$ để quyết định thứ tự. Counting Sort **không so sánh** – nó đếm số lần xuất hiện của mỗi giá trị rồi tính vị trí. Vì không thuộc mô hình "decision tree" nên không bị ràng buộc bởi $\Omega(n \lg n)$.

Đánh đổi: Counting Sort cần biết phạm vi giá trị k và tốn $\Theta(k)$ bộ nhớ phụ. Nếu $k \gg n$ (ví dụ: sắp xếp $n = 100$ số trong khoảng $[1, 10^9]$), Counting Sort không khả thi.

12.3 Chương 4: Lý thuyết Đồ thị

12.3.1 Tại sao? 4.1 – Ma trận kề tốn $\Theta(V^2)$ dù đồ thị thưa?

Câu hỏi: Ma trận kề tốn $\Theta(V^2)$ bộ nhớ kể cả khi đồ thị rất thưa ($E \ll V^2$). Tại sao? Và tại sao danh sách kề chỉ tốn $\Theta(V + E)$?

Trả lời:

- **Ma trận kề:** Luôn cấp phát bảng $V \times V$, bất kể có bao nhiêu cạnh. Với 5 đỉnh, 3 cạnh: ta vẫn tạo bảng $5 \times 5 = 25$ ô, trong đó chỉ 3 ô $= 1$, còn lại 22 ô $= 0 \rightarrow$ **lãng phí**.
- **Danh sách kề:** Mỗi đỉnh có 1 danh sách chỉ chứa các đỉnh kề. Tổng $= V$ danh sách $+ 2E$ phần tử (đồ thị vô hướng) hoặc E phần tử (có hướng) $= \Theta(V + E)$.

Ví dụ: 5 đỉnh, 3 cạnh $\{(1, 2), (2, 3), (3, 4)\}$:

- Ma trận: $5 \times 5 = 25$ ô, chỉ 6 ô $\neq 0$.
- Danh sách kề: 5 heads + 6 entries = 11 phần tử. Tiết kiệm hơn hẳn!

12.4 Chương 5: Greedy – MST

12.4.1 Tại sao? 5.1 – Cạnh nhẹ nhất qua cut phải thuộc MST?

Câu hỏi: Tại sao cạnh nhẹ nhất qua một cut **phải** thuộc MST? (Cut Property)

Trả lời (phản chứng):

1. Giả sử (u, v) là cạnh nhẹ nhất qua cut $(S, V \setminus S)$, nhưng $(u, v) \notin T^*$ (MST tối ưu).
2. Thêm (u, v) vào $T^* \rightarrow$ tạo đúng 1 chu trình (vì cây + 1 cạnh = 1 cycle).
3. Trong chu trình này, tồn tại cạnh khác (x, y) cũng qua cut $(S, V \setminus S)$ (vì chu trình phải "quay về" phía bên kia cut).
4. Vì (u, v) là cạnh **nhẹ nhất** qua cut: $w(u, v) \leq w(x, y)$.
5. Bỏ (x, y), giữ $(u, v) \rightarrow$ tạo cây $T' = T^* - \{(x, y)\} + \{(u, v)\}$ có tổng trọng số $\leq w(T^*)$.
6. Mà T^* là MST $\rightarrow w(T') = w(T^*) \rightarrow T'$ cũng là MST và chứa (u, v). **Mâu thuẫn** với giả thiết ban đầu. $\square$

12.4.2 Tự Trace 5.1 – Kruskal: Trace đồ thị 4 đỉnh

Câu hỏi: Cho đồ thị 4 đỉnh $\{1, 2, 3, 4\}$ với cạnh: $(1, 2, 4)$, $(1, 3, 1)$, $(2, 3, 3)$, $(2, 4, 2)$, $(3, 4, 5)$.

Trả lời:

Sắp xếp cạnh tăng dần: $(1,3,1)$, $(2,4,2)$, $(2,3,3)$, $(1,2,4)$, $(3,4,5)$.

Bước	**Cạnh**	w	**Union-Find**	**Hành động**
1	$(1,3)$	1	$\{1,3\},\{2\},\{4\}$	Thêm vào MST
2	$(2,4)$	2	$\{1,3\},\{2,4\}$	Thêm vào MST
3	$(2,3)$	3	$\{1,2,3,4\}$	Thêm vào MST
4	$(1,2)$	4	–	Bỏ (chu trình)
5	$(3,4)$	5	–	Bỏ (chu trình)

MST: $\{(1,3),(2,4),(2,3)\}$, tổng $= 1+2+3 = \boxed{6}$.

12.5 Chương 6: Dijkstra

12.5.1 Tại sao? 6.1 – Dijkstra không hoạt động với cạnh âm?

Câu hỏi: Tại sao Dijkstra không hoạt động với cạnh có trọng số âm? Xây dựng phản ví dụ.

Trả lời:

Phản ví dụ: 3 đỉnh $\{s,a,b\}$, cạnh: $s \to a(1)$, $s \to b(3)$, $b \to a(-4)$.

Bước	$d[s]$	$d[a]$	$d[b]$	**Extract**	**Ghi chú**
Init	**[0]**	∞	∞	s	–
1	0	**[1]**	3	a	$d[a]=1$, $d[b]=3$
2	0	1	**[3]**	b	$b \to a$: $3+(-4)=-1<1$, nhưng a đã "khóa"!

Dijkstra cho $d[a]=1$, nhưng đường đi thực sự ngắn nhất là $s \to b \to a = 3+(-4) = -1$.

Nguyên nhân: Dijkstra giả định rằng khi đỉnh u được "khóa" (extracted from Q), $d[u]$ là final. Với cạnh âm, giả định này **sai** – có thể tìm đường ngắn hơn qua đỉnh chưa xử lý. Giải pháp: dùng **Bellman-Ford** ($O(VE)$).

12.5.2 Tự Trace 6.1 – Dijkstra: Trace bảng khoảng cách

Câu hỏi: Cho đồ thị 4 đỉnh $\{s,a,b,t\}$ với cạnh: $s{\to}a(1)$, $s{\to}b(4)$, $a{\to}b(2)$, $a{\to}t(6)$, $b{\to}t(3)$.

Trả lời:

Bước	$d[s]$	$d[a]$	$d[b]$	$d[t]$	**ExtractMin**
Init	**[0]**	∞	∞	∞	s
1	0	**[1]**	4	∞	a
2	0	1	**[3]**	7	b
3	0	1	3	**[6]**	t

Chi tiết Relaxation:

- Bước 1: Extract s. Relax $s{\to}a$: $d[a]=\min(\infty,0+1)=1$. Relax $s{\to}b$: $d[b]=\min(\infty,0+4)=4$.
- Bước 2: Extract a. Relax $a{\to}b$: $d[b]=\min(4,1+2)=3$. Relax $a{\to}t$: $d[t]=\min(\infty,1+6)=7$.
- Bước 3: Extract b. Relax $b{\to}t$: $d[t]=\min(7,3+3)=6$.

Đường đi ngắn nhất $s \to t$: $s \to a \to b \to t$, chi phí $= 1+2+3 = \boxed{6}$.

12.6 Chương 7: Quy hoạch động – Floyd-Warshall

12.6.1 Tại sao? 7.1 – Floyd-Warshall: vòng lặp k phải ở ngoài cùng?

Câu hỏi: Tại sao vòng lặp k phải ở ngoài cùng? Nếu đặt i hoặc j ở ngoài thì sao?

Trả lời: Công thức Floyd-Warshall:

$$D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)$$

Ý nghĩa: $D^{(k)}[i][j]$ = đường đi ngắn nhất từ i đến j **chỉ dùng đỉnh trung gian trong tập** $\{1,2,\ldots,k\}$.

- k **ở ngoài cùng:** Mỗi lần tăng k, ta "mở khóa" thêm 1 đỉnh trung gian. Toàn bộ bảng $D^{(k)}$ được tính từ $D^{(k-1)}$ đã hoàn chỉnh → **đúng**.
- **Nếu** i **ở ngoài:** Khi tính $D[i][j]$ với k nào đó, bảng $D[\cdot][k]$ và $D[k][\cdot]$ có thể chưa được cập nhật đầy đủ ở tầng k hiện tại → sử dụng giá trị sai → **kết quả sai**.

Trực giác: "Mở khóa" từng đỉnh trung gian theo thứ tự = xây dựng DP bottom-up đúng cách. Đảo thứ tự vòng lặp = phá vỡ thứ tự phụ thuộc DP.

12.7 Chương 9: P, NP, NP-Complete

12.7.1 Tại sao? 9.1 – Tại sao một số bài toán không giải được trong thời gian đa thức?

Câu hỏi: Tại sao một số bài toán không thể giải được trong thời gian đa thức? Nếu P $\neq$ NP, điều đó có ý nghĩa gì?

Trả lời:

1. Trực giác – "Kiểm tra nhanh $\neq$ Tìm nhanh":

- Cho bạn 1 bài Sudoku đã giải → **kiểm tra** đúng/sai rất nhanh (chỉ cần xem hàng, cột, ô 3×3).
- Nhưng **tìm** lời giải từ bảng trống? Phải thử nhiều tổ hợp → không ai biết cách nào nhanh hơn brute force.
- Đây chính là bản chất của **NP**: kiểm tra lời giải trong $O(n^k)$, nhưng tìm lời giải có thể cần $O(2^n)$.

2. Tại sao tin rằng P $\neq$ NP?

- Hàng ngàn bài NP-Complete đã được phát hiện từ năm 1971 (Cook-Levin: SAT). **Không ai** tìm được thuật toán polynomial cho bất kỳ bài nào.
- Nếu giải được **1 bài** NPC trong polynomial time → **mọi bài NP** đều giải được nhanh (vì reduction). Điều này "quá tốt để tin".
- Tuy nhiên, P $\neq$ NP vẫn là **giả thuyết** – chưa ai chứng minh được (1 trong 7 bài toán Millennium Prize, thưởng $1 triệu USD).

3. Ý nghĩa thực tiễn:

- Nếu gặp bài NP-Complete → **đừng cố tìm thuật toán exact** $O(n^k)$. Thay vào đó:
  1. Dùng **approximation algorithm** (nghiệm gần đúng, đảm bảo chất lượng).
  2. Dùng **heuristic** (greedy, genetic algorithm... – không đảm bảo nhưng thực tế tốt).
  3. Giải trên **input nhỏ** bằng backtracking hoặc DP mũ.
- Mật mã học (RSA) dựa vào giả thuyết P $\neq$ NP: phân tích thừa số nguyên tố là "khó". Nếu P = NP → mọi hệ mật mã hiện tại bị phá.

13 Chương 13: Phụ lục – Bảng tên gọi Ký hiệu Toán học

Bảng dưới đây liệt kê tất cả các ký hiệu toán học xuất hiện trong tài liệu, kèm tên gọi tiếng Việt, cách đọc, và ý nghĩa trong ngữ cảnh thuật toán.

13.1 Ký hiệu tiệm cận (Asymptotic Notation)

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$\Theta(f)$	Theta	"Tê-ta của f"	Chặn chặt (tight bound)
$O(f)$	Big-O	"Ô lớn của f"	Chặn trên (upper bound)
$\Omega(f)$	Omega	"Ô-mê-ga của f"	Chặn dưới (lower bound)
$o(f)$	Little-o	"ô nhỏ của f"	Chặn trên không chặt
$\omega(f)$	Little-omega	"ô-mê-ga nhỏ của f"	Chặn dưới không chặt
$\lg n$	Logarit cơ số 2	"lóc cơ số 2 của n"	$= \log_2 n$
$\log_b n$	Logarit cơ số b	"lóc cơ số b của n"	Logarit tổng quát

13.2 Ký hiệu quan hệ và so sánh

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$=$	Bằng	"bằng"	Hai vế bằng nhau
$\neq$	Khác	"khác"	Hai vế không bằng nhau
$<$	Nhỏ hơn	"nhỏ hơn"	Bé hơn nghiêm ngặt
$>$	Lớn hơn	"lớn hơn"	Lớn hơn nghiêm ngặt
$\leq$	Nhỏ hơn hoặc bằng	"nhỏ hơn hoặc bằng"	Bé hơn hoặc bằng
$\geq$	Lớn hơn hoặc bằng	"lớn hơn hoặc bằng"	Lớn hơn hoặc bằng
$\ll$	Nhỏ hơn rất nhiều	"nhỏ hơn rất nhiều"	Ví dụ: $E \ll V^2$
$\approx$	Xấp xỉ	"xấp xỉ"	Gần bằng

13.3 Ký hiệu tập hợp và logic

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$\in$	Thuộc	"thuộc"	Phần tử thuộc tập hợp
$\notin$	Không thuộc	"không thuộc"	Phần tử không thuộc tập
$\subseteq$	Tập con	"là tập con của"	$A \subseteq B$: mọi phần tử A đều trong B
$\forall$	Với mọi	"với mọi"	Lượng từ phổ quát
$\exists$	Tồn tại	"tồn tại"	Lượng từ tồn tại
$\cup$	Hợp	"hợp"	Hợp hai tập $A \cup B$
$\cap$	Giao	"giao"	Giao hai tập $A \cap B$
$\emptyset$	Tập rỗng	"tập rỗng"	Tập không có phần tử
$\lvert S \rvert$	Lực lượng	"lực lượng của S"	Số phần tử của tập S
$\{\ \}$	Ngoặc nhọn	"tập"	Ký hiệu liệt kê tập hợp
$\mathbb{R}^+$	Số thực dương	"tập số thực dương"	Trọng số cạnh thường $\in \mathbb{R}^+$

13.4 Ký hiệu mũi tên và suy ra

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$\rightarrow$	Mũi tên phải	"suy ra" / "đến"	Ánh xạ, hoặc hệ quả
$\Leftrightarrow$	Tương đương	"khi và chỉ khi"	Điều kiện cần và đủ
$\Longleftrightarrow$	Tương đương	"khi và chỉ khi"	Dạng dài của $\Leftrightarrow$
$\rightsquigarrow$	Đường đi	"đi đến"	Đường đi trong đồ thị
$\Rightarrow$	Suy ra	"suy ra"	Nếu A thì B

13.5 Ký hiệu toán học tổng quát

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$\sum$	Tổng	"tổng"	Phép cộng nhiều số hạng
min	Giá trị nhỏ nhất	"min"	Cực tiểu
max	Giá trị lớn nhất	"max"	Cực đại
∞	Vô cực	"vô cực"	Giá trị lớn vô hạn
$\lfloor x \rfloor$	Phần nguyên dưới	"sàn của x"	Số nguyên lớn nhất $\leq x$
$\lceil x \rceil$	Phần nguyên trên	"trần của x"	Số nguyên nhỏ nhất $\geq x$
$n!$	Giai thừa	"n giai thừa"	$= 1 \times 2 \times \ldots \times n$
$\sqrt{x}$	Căn bậc hai	"căn của x"	Số mà bình phương bằng x
$\langle a, b \rangle$	Ngoặc nhọn / Bộ	"bộ a, b"	Dãy có thứ tự
$\cdot$	Nhân	"nhân"	Phép nhân: $a \cdot b$
$\times$	Nhân / Kích thước	"nhân" / "cross"	Ma trận $m \times n$
φ	Phi	"phi"	Tỷ lệ vàng ≈ 1.618
ε	Epsilon	"ép-xi-lon"	Số dương rất nhỏ
δ	Delta	"đen-ta"	Khoảng cách ngắn nhất $\delta(s, v)$
α	Alpha	"an-pha"	Hàm nghịch Ackermann (Union-Find)
π hoặc $\pi[v]$	Pi	"pi"	Đỉnh cha trên đường đi ngắn nhất
ℓ	Ell	"eo"	Hàm trọng số $\ell : E \to \mathbb{R}^+$

13.6 Ký hiệu đồ thị

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$G = (V, E)$	Đồ thị	"đồ thị G"	V: tập đỉnh, E: tập cạnh
V	Tập đỉnh	"V"	Vertex set
E	Tập cạnh	"E"	Edge set
(u, v)	Cạnh	"cạnh u v"	Cạnh nối đỉnh u và v
$\deg(v)$	Bậc	"bậc của v"	Số cạnh kề đỉnh v
$w(u, v)$	Trọng số	"trọng số cạnh u v"	Chi phí / độ dài cạnh
$d[v]$	Khoảng cách ước lượng	"d của v"	Ước lượng trên $\delta(s, v)$
$\delta(s, v)$	Khoảng cách ngắn nhất	"delta s v"	Đường đi ngắn nhất thực sự
$\text{Adj}[u]$	Danh sách kề	"kề của u"	Các đỉnh kề với u

13.7 Ký hiệu reduction / NP

Ký hiệu	Tên tiếng Việt	Cách đọc	Ý nghĩa
$\leq_p$	Quy về đa thức	"quy về (trong đa thức)"	$A \leq_p B$: A quy về B
P	Lớp P	"lớp P"	Giải được trong polynomial
NP	Lớp NP	"lớp NP"	Kiểm tra được trong polynomial
NPC	NP-Complete	"NP đầy đủ"	Khó nhất trong NP

Table 14: Ý nghĩa chiều Reduction trong chứng minh NP-Complete

---

13.8 Tóm tắt: Greedy vs D&C vs DP

	Greedy	**D&C**	**DP**
Bài con	Giảm dần	Tách biệt	Chồng lấp
Quyết định	Trước (không quay lui)	Sau (gộp)	Sau (bảng)
Time	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$–$O(n^3)$
Ví dụ	Kruskal, Dijkstra	Merge Sort, Binary Search	Floyd, MCM

Table 15: Quick Reference: Ba phương pháp thiết kế

Lecture Notes theo phong cách CLRS – Introduction to Algorithms, 4th Edition
Cormen, Leiserson, Rivest, Stein
Tài liệu ôn thi | Ngày tạo: 12/03/2026